```
/* ========== ■■■ App ■■■■ ========== */
/* ■■■■■■ Flutter ■■■■ */
/* ■■■V1.0.0 */


/* === lib/presentation/pages/chat/auction_chat_page.dart === */
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:cached_network_image/cached_network_image.dart';
import 'package:flutter/material.dart';
import 'package:flutter_html/flutter_html.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:image_picker/image_picker.dart';
import 'package:shared_preferences/shared_preferences.dart';

import '../../../data/models/announce_model.dart';
import '../../../data/models/auction_item_model.dart';
import '../../../data/models/chat_message_model.dart';
import '../../../data/repositories/announce_repository.dart';
import '../../../data/repositories/auth_repository.dart';
import '../../../data/repositories/chat_repository.dart';
import '../../../data/repositories/friend_repository.dart';
import '../../../data/repositories/user_repository.dart';
import '../../../data/services/notification_permission_service.dart';
import '../../../data/services/upload_service.dart';
import '../../../data/services/wechat_service.dart';
import '../../../core/widgets/app_bottom_sheet.dart';
import '../../providers/auction_provider.dart';
import '../../providers/chat_emoji_store.dart';
import '../../providers/chat_store.dart';
import '../../providers/user_provider.dart';
import '../../widgets/chat/chat_input_area.dart';
import '../../widgets/chat/messages/message_widget.dart';
import '../../widgets/common/user_profile_dialog.dart';
import '../announce/announce_list_page.dart';

/// ■■■■■■■■■■■
/// ■ UniApp auction.vue ■■■■
class AuctionChatPage extends ConsumerStatefulWidget {
  const AuctionChatPage({super.key});

  @override
  ConsumerState<AuctionChatPage> createState() => _AuctionChatPageState();
}

class _AuctionChatPageState extends ConsumerState<AuctionChatPage>
    with TickerProviderStateMixin {
  final ScrollController _scrollController = ScrollController();
```

```
  final ImagePicker _imagePicker = ImagePicker();
  final UploadService _uploadService = UploadService();
  final AnnounceRepository _announceRepository = AnnounceRepository();

  // UI ■■
  bool _showAuctionList = false;
  bool _showUnreadNotification = false;
  bool _isUploadingImage = false;
  bool _isLoadingMessages = false;

  // ■■■■■■
  AnnounceDto? _currentAnnounce;
  bool _showAnnouncementDialog = false;
  bool _isShowingAnnouncementDialog = false; // ■■■■■■
  static const String _auctionNoticeKey = 'auctionNotice';
  static const int _announceCategoryId = 2;

  // ■■■■■■■■
  bool _isLoadingHistory = false;
  bool _hasMoreHistory = true;

  // ■■■■■
  late AnimationController _auctionListAnimationController;
  late AnimationController _unreadAnimationController;
  late Animation<Offset> _auctionListAnimation;
  late Animation<Offset> _unreadAnimation;

  static const String _channel = '-1_auction';
  static const int _channelId = -1;
  static const String _channelName = '■■■';

  @override
  void initState() {
    super.initState();
    _scrollController.addListener(_onScroll);

    _auctionListAnimationController = AnimationController(
      duration: const Duration(milliseconds: 300),
      vsync: this,
    );
    _auctionListAnimation =
        Tween<Offset>(begin: const Offset(1.0, 0.0), end: Offset.zero).animate(
          CurvedAnimation(
            parent: _auctionListAnimationController,
            curve: Curves.easeOut,
          ),
        );

    _unreadAnimationController = AnimationController(
      duration: const Duration(milliseconds: 300),
```

```
      vsync: this,
    );
    _unreadAnimation =
        Tween<Offset>(begin: const Offset(1.0, 0.0), end: Offset.zero).animate(
          CurvedAnimation(
            parent: _unreadAnimationController,
            curve: Curves.easeOut,
          ),
        );

    WidgetsBinding.instance.addPostFrameCallback((_) async {
      ref.read(auctionProvider.notifier).loadAuctions();

      await ref.read(chatStoreProvider.notifier).connectServer();

      ref
          .read(chatStoreProvider.notifier)
          .setCurrentChatId(_channelId, name: _channelName);

      await ref.read(chatStoreProvider.notifier).joinChannel(_channel);

      // ■■■■■■■■
      _loadAnnouncement();

      setState(() => _isLoadingMessages = true);
      await ref.read(chatStoreProvider.notifier).getGroupHistory(_channel);
      setState(() => _isLoadingMessages = false);

      // ■■■■■■
      _scrollToBottom();

      // ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
      ref.listenManual<List<ChatMessage>>(currentChatMessagesProvider, (
        previous,
        next,
      ) {
        print('[AuctionChat] ========== ref.listen ■■ (initState) ==========');
        print('[AuctionChat] previous.length = ${previous?.length}');
        print('[AuctionChat] next.length = ${next.length}');
        print('[AuctionChat] _isLoadingHistory = $_isLoadingHistory');

        // ■■■■■■■■■■■■■■■■■■■■■■■■
        if (next.length > previous!.length && !_isLoadingHistory) {
          print('[AuctionChat] ■ ■■■■■■■■■■■■■■■');
          // ■■■■■■■■■■■■■■■■■■■■■■■
          if (mounted && !_isLoadingMessages) {
            // ■■■■■■■■■■■■■■■■■■■■
            Future.delayed(const Duration(milliseconds: 100), () {
              if (mounted) {
                _scrollToBottom();
```

```
              }
            });
          }
        } else {
          print('[AuctionChat] ■ ■■■■■■■■');
        }
      });
    });
  }

  @override
  void dispose() {
    _scrollController.removeListener(_onScroll);
    _scrollController.dispose();
    _auctionListAnimationController.dispose();
    _unreadAnimationController.dispose();
    super.dispose();
  }

  void _onScroll() {
    // ■■■■■■■■■■■
    if (_scrollController.position.pixels <= 0 &&
        !_isLoadingHistory &&
        _hasMoreHistory) {
      final messages = ref.read(currentChatMessagesProvider);
      if (messages.isNotEmpty) {
        final firstTime = messages.first.time;
        if (firstTime != null) {
          setState(() {
            _isLoadingHistory = true;
          });

          ref
              .read(chatStoreProvider.notifier)
              .getGroupHistory(_channel, lastTime: firstTime)
              .then((_) {
                if (mounted) {
                  setState(() {
                    _isLoadingHistory = false;
                    // ■■■■■■■■■■■■■■■■■■■■■■■■
                  });
                }
              })
              .catchError((error) {
                if (mounted) {
                  setState(() {
                    _isLoadingHistory = false;
                  });
                }
              });
```

```
        }
      }
    }
  }

  void _scrollToBottom() {
    print('[AuctionChat] _scrollToBottom() ■■■');
    print(
      '[AuctionChat] _scrollController.hasClients = ${_scrollController.hasClients}',
    );

    // ■■■■■■■■■■■ - ■■■■■■■■■■■■■■■
    WidgetsBinding.instance.addPostFrameCallback((_) {
      Future.delayed(const Duration(milliseconds: 100), () {
        if (!mounted) {
          print('[AuctionChat] ■ ■■■■■■■■■■■');
          return;
        }

        if (!_scrollController.hasClients) {
          print('[AuctionChat] ■ hasClients = false■■■■■■');
          return;
        }

        try {
          final currentPixels = _scrollController.position.pixels;
          final extent = _scrollController.position.maxScrollExtent;
          final viewportDimension =
              _scrollController.position.viewportDimension;

          print('[AuctionChat] ========== ■■■■■■■ ==========');
          print('[AuctionChat] ■■■■■ pixels = $currentPixels');
          print('[AuctionChat] ■■■■■■■ maxScrollExtent = $extent');
          print('[AuctionChat] ■■■■■ viewportDimension = $viewportDimension');
          print('[AuctionChat] ■■■■■■■ = ${currentPixels < extent}');
          print('[AuctionChat] ====================================');

          // ■■■■■■■■■■■■■■■
          if (currentPixels < extent) {
            _scrollController.animateTo(
              extent,
              duration: const Duration(milliseconds: 300),
              curve: Curves.easeOut,
            );
            print('[AuctionChat] ■ ■■■■ animateTo ■■■■ $extent');
          } else {
            print('[AuctionChat] ■■ ■■■■■■■■■■■■■■■');
          }
        } catch (e, stackTrace) {
          print('[AuctionChat] ■ ■■■■■: $e');
```

```
          print('[AuctionChat] StackTrace: $stackTrace');
        }
      });
    });
    print('[AuctionChat] ■■■■■■■■■■■■■■■');
  }

  Future<void> _onSendText(String text) async {
    await ref
        .read(chatStoreProvider.notifier)
        .sendChannelMsg(
          channel: _channel,
          message: text,
          type: ChatMessageType.text,
        );
    _scrollToBottom();
  }

  Future<void> _onPickImage() async {
    try {
      final XFile? image = await _imagePicker.pickImage(
        source: ImageSource.gallery,
        maxWidth: 1024,
        maxHeight: 1024,
        imageQuality: 85,
      );

      if (image != null) {
        // ■■■■■■■ ID
        final userId = _getCurrentUserId();

        // ■■■■■■■■
        setState(() {
          _isUploadingImage = true;
        });

        try {
          // ■■■■
          final imageUrl = await _uploadService.uploadImage(
            image.path,
            userId: userId?.toString(),
          );

          if (imageUrl != null) {
            // ■■■■■■■
            final file = File(image.path);
            final bytes = await file.readAsBytes();
            final decodedImage = await decodeImageFromList(bytes);
            final width = decodedImage.width;
            final height = decodedImage.height;
```

```
301
302             // ■■ payload■■ UniApp ■■■■
303             final payload = {'url': imageUrl, 'width': width, 'height': height};
304
305             // ■■■■■■
306             await ref
307                 .read(chatStoreProvider.notifier)
308                 .sendChannelMsg(
309                   channel: _channel,
310                   message: imageUrl,
311                   type: ChatMessageType.image,
312                   payload: payload,
313                 );
314             _scrollToBottom();
315           } else {
316             if (mounted) {
317               ScaffoldMessenger.of(
318                 context,
319               ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■■')));
320             }
321           }
322         } finally {
323           // ■■■■■■
324           if (mounted) {
325             setState(() {
326               _isUploadingImage = false;
327             });
328           }
329         }
330       }
331     } catch (e) {
332       if (mounted) {
333         setState(() {
334           _isUploadingImage = false;
335         });
336         ScaffoldMessenger.of(
337           context,
338         ).showSnackBar(SnackBar(content: Text('■■■■■■■$e')));
339       }
340     }
341   }
342
343   /// ■■■■■■■■ UniApp calculateMinBidPrice ■■■■■
344   int _calculateMinBidPrice(double currentPrice, bool isKasec) {
345     int minPrice = 5;
346
347     if (currentPrice > 0) {
348       if (currentPrice < 100) {
349         minPrice = (currentPrice + 5).toInt();
350       } else if (currentPrice < 1000) {
```

```
        minPrice = (currentPrice + 5).toInt();
      } else if (currentPrice < 2000) {
        minPrice = (currentPrice + 10).toInt();
      } else if (currentPrice < 5000) {
        minPrice = (currentPrice + 20).toInt();
      } else if (currentPrice < 10000) {
        minPrice = (currentPrice + 50).toInt();
      } else {
        minPrice = (currentPrice + 100).toInt();
      }
    }

    // ■■■■■■■■■■■■■■■■■■3■
    if (isKasec) {
      minPrice = (currentPrice + (minPrice - currentPrice) * 3).toInt();
    }

    return minPrice;
  }

  /// ■■■■■■■■■■ UniApp auction.vue bid() ■■■
  Future<void> _showBidDialog() async {
    final auctionState = ref.read(auctionProvider);
    final onAuctionItem = auctionState.onAuctionItem;

    if (onAuctionItem == null || onAuctionItem.id == null) {
      ScaffoldMessenger.of(
        context,
      ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■■')));
      return;
    }

    final auctionItemId = onAuctionItem.id!;

    // ■■■■■■
    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (context) => const Center(child: CircularProgressIndicator()),
    );

    try {
      // 1. ■■■■■■■■■■■■
      final auctionItemDetail = await ref
          .read(auctionProvider.notifier)
          .getAuctionDetail(auctionItemId);

      if (auctionItemDetail == null) {
        if (mounted) {
          Navigator.pop(context); // ■■■■■
```

```
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■■■■■■■')));
        }
        return;
      }

      // 2. ■■■■■■■
      if (auctionItemDetail.status != AuctionStatusEnum.auctioning) {
        if (mounted) {
          Navigator.pop(context); // ■■■■■■
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('■■■■■■■■')));
        }
        return;
      }

      // 3. ■■■■■■■
      final isKasecMode = await ref
          .read(auctionProvider.notifier)
          .syncKasecStatus(auctionItemId);

      // 4. ■■■■■■■
      final currentPrice =
          auctionItemDetail.currentPrice ??
          auctionItemDetail.startingPrice ??
          0;
      final minPrice = _calculateMinBidPrice(currentPrice, isKasecMode);

      if (!mounted) return;
      Navigator.pop(context); // ■■■■■■

      // 5. ■■■■■■■■
      final TextEditingController priceController = TextEditingController();

      showDialog(
        context: context,
        builder: (dialogContext) => Dialog(
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
          child: Container(
            padding: const EdgeInsets.all(20),
            child: Column(
              mainAxisSize: MainAxisSize.min,
              children: [
                // ■■
                Text(
                  isKasecMode ? '■■■■ - ■■■■■■' : '■■',
                  style: const TextStyle(
                    fontSize: 18,
```

```
                    fontWeight: FontWeight.bold,
                    color: Color(0xFF333333),
                  ),
                ),
                const SizedBox(height: 16),

                // ■■■■■■
                if (isKasecMode)
                  Container(
                    padding: const EdgeInsets.all(8),
                    decoration: BoxDecoration(
                      color: const Color(0xFFFFF5F5),
                      border: Border.all(color: Colors.red),
                      borderRadius: BorderRadius.circular(4),
                    ),
                    child: Text(
                      '■■■■■■■■■■■■■■■■■■\n■■■■■■$minPrice R■',
                      style: const TextStyle(color: Colors.red, fontSize: 14),
                      textAlign: TextAlign.center,
                    ),
                  )
                else
                  Text(
                    '■■■■ $minPrice R',
                    style: const TextStyle(color: Color(0xFF666666), fontSize: 14),
                  ),

                const SizedBox(height: 16),

                // ■■■
                TextField(
                  controller: priceController,
                  keyboardType: TextInputType.number,
                  textAlign: TextAlign.center,
                  style: const TextStyle(fontSize: 16),
                  decoration: InputDecoration(
                    hintText: '■■■■■■■',
                    suffixText: 'R',
                    isDense: true,
                    contentPadding: const EdgeInsets.symmetric(horizontal: 12, vertical: 12),
                    border: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(8),
                      borderSide: const BorderSide(color: Color(0xFFDDDDDD)),
                    ),
                    enabledBorder: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(8),
                      borderSide: const BorderSide(color: Color(0xFFDDDDDD)),
                    ),
                    focusedBorder: OutlineInputBorder(
                      borderRadius: BorderRadius.circular(8),
```

```
                      borderSide: const BorderSide(color: Color(0xFF007AFF)),
                    ),
                  ),
                ),

                const SizedBox(height: 20),

                // ■■■■
                Row(
                  children: [
                    // ■■■■
                    Expanded(
                      child: GestureDetector(
                        onTap: () => Navigator.pop(dialogContext),
                        child: Container(
                          height: 40,
                          alignment: Alignment.center,
                          decoration: BoxDecoration(
                            color: const Color(0xFFF5F5F5),
                            borderRadius: BorderRadius.circular(8),
                          ),
                          child: const Text(
                            '■■',
                            style: TextStyle(fontSize: 16, color: Color(0xFF333333)),
                          ),
                        ),
                      ),
                    ),
                    const SizedBox(width: 15),
                    // ■■■■
                    Expanded(
                      child: GestureDetector(
                        onTap: () async {
                          final price = int.tryParse(priceController.text);
                          if (price == null) {
                            ScaffoldMessenger.of(dialogContext).showSnackBar(
                              const SnackBar(content: Text('■■■■■')),
                            );
                            return;
                          }

                          if (price < 5) {
                            ScaffoldMessenger.of(dialogContext).showSnackBar(
                              const SnackBar(content: Text('■■■■■5R■■■■■■')),
                            );
                            return;
                          }

                          if (isKasecMode && price < minPrice) {
                            ScaffoldMessenger.of(dialogContext).showSnackBar(
```

```
                              SnackBar(content: Text('■■■■■■■■■■■■■■■■■$minPrice')),
                            );
                            return;
                          }

                          Navigator.pop(dialogContext);

                          final success = await ref
                              .read(auctionProvider.notifier)
                              .bid(auctionItemId, price.toDouble());

                          if (mounted) {
                            ScaffoldMessenger.of(context).showSnackBar(
                              SnackBar(
                                content: Text(success ? '■■■■: $price R' : '■■■■■■■■■'),
                              ),
                            );
                            if (success) {
                              ref.read(auctionProvider.notifier).loadAuctions();
                            }
                          }
                        },
                        child: Container(
                          height: 40,
                          alignment: Alignment.center,
                          decoration: BoxDecoration(
                            gradient: isKasecMode
                                ? const LinearGradient(
                                    colors: [Color(0xFFFF7144), Color(0xFFFF9500)],
                                  )
                                : const LinearGradient(
                                    colors: [Color(0xFF007AFF), Color(0xFF0056CC)],
                                  ),
                            borderRadius: BorderRadius.circular(8),
                          ),
                          child: const Text(
                            '■■',
                            style: TextStyle(fontSize: 16, color: Colors.white, fontWeight: FontWeig
                          ),
                        ),
                      ),
                    ),
                  ),
                ],
              ),
            ],
          ),
        ),
      );
    } catch (e) {
```

```
      if (mounted) {
        Navigator.pop(context); // ■■■■■
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■■■■■■■■')));
      }
    }
  }

  void _toggleAuctionList() {
    setState(() {
      _showAuctionList = !_showAuctionList;
    });

    if (_showAuctionList) {
      _auctionListAnimationController.forward();
    } else {
      _auctionListAnimationController.reverse();
    }
  }

  @override
  Widget build(BuildContext context) {
    final messages = ref.watch(currentChatMessagesProvider);
    final chatState = ref.watch(chatStoreProvider);
    final auctionState = ref.watch(auctionProvider);
    final onAuctionItem = auctionState.onAuctionItem;

    // ■■■■■■■■ - ■■■■■■■■
    if (_showAnnouncementDialog &&
        _currentAnnounce != null &&
        !_isShowingAnnouncementDialog) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        if (_showAnnouncementDialog &&
            mounted &&
            !_isShowingAnnouncementDialog) {
          _displayAnnouncementDialog();
        }
      });
    }

    return Scaffold(
      appBar: AppBar(
        title: Text(
          _channelName,
          style: const TextStyle(fontSize: 20, color: Colors.white),
        ),
        backgroundColor: const Color(0xFFF4835A),
        foregroundColor: Colors.white,
      ),
```

```
      body: Stack(
        children: [
          Column(
            children: [
              _buildAnnouncementBar(),
              Expanded(
                child: Container(
                  color: const Color(0xFFFAF1F0),
                   child: _buildUnifiedMessageList(messages),
                ),
              ),
              ChatInputArea(
                onSendText: _onSendText,
                onSelectImage: _onPickImage,
                showFavoriteTab: true,
                onSelectFavoriteEmoji: _onSelectFavoriteEmoji,
              ),
            ],
          ),
          _buildRightSideButtons(onAuctionItem),
          if (_showUnreadNotification)
            _buildNewMessageButton(chatState.unreadCount),
          if (_showAuctionList) _buildAuctionListPanel(auctionState),
          // ■■■■
          if (_isUploadingImage) _buildLoadingOverlay(),
          if (_isLoadingMessages) _buildMessageLoadingOverlay(),
        ],
      ),
    );
  }

  Widget _buildMessageLoadingOverlay() {
    return Container(
      color: Colors.black.withOpacity(0.3),
      child: Center(
        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 20),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(12),
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const SizedBox(
                width: 32,
                height: 32,
                child: CircularProgressIndicator(
                  strokeWidth: 3,
                  valueColor: AlwaysStoppedAnimation<Color>(Color(0xFFF4835A)),
```

```
                ),
              ),
              const SizedBox(height: 16),
              Text(
                '■■■■■■...',
                style: TextStyle(fontSize: 14, color: Colors.grey.shade700),
              ),
            ],
          ),
        ),
      ),
    );
  }

  Widget _buildLoadingOverlay() {
    return Container(
      color: Colors.black.withOpacity(0.3),
      child: Center(
        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 20),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(12),
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const SizedBox(
                width: 32,
                height: 32,
                child: CircularProgressIndicator(
                  strokeWidth: 3,
                  valueColor: AlwaysStoppedAnimation<Color>(Color(0xFFF4835A)),
                ),
              ),
              const SizedBox(height: 16),
              Text(
                '■■■■■■...',
                style: TextStyle(fontSize: 14, color: Colors.grey.shade700),
              ),
            ],
          ),
        ),
      ),
    );
  }

  Widget _buildRightSideButtons(dynamic onAuctionItem) {
    return Positioned(
      right: 0,
```

```
      top: 100,
      child: Column(
        children: [
          GestureDetector(
            onTap: _toggleAuctionList,
            child: Container(
              padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 12),
              decoration: const BoxDecoration(
                color: Color(0xFFFF7144),
                borderRadius: BorderRadius.only(
                  topLeft: Radius.circular(8),
                  bottomLeft: Radius.circular(8),
                ),
              ),
              child: const Text(
                '■■■',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 14,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ),
          ),
          const SizedBox(height: 12),
          if (onAuctionItem != null && onAuctionItem.id != null) ...[
            GestureDetector(
              onTap: () => _showAuctionDetail(onAuctionItem),
              child: Container(
                padding: const EdgeInsets.symmetric(horizontal: 8, vertical: 8),
                decoration: BoxDecoration(
                  color: const Color(0xFFFF7144),
                  borderRadius: BorderRadius.circular(8),
                ),
                child: const Text(
                  '■■■■',
                  style: TextStyle(
                    color: Colors.white,
                    fontSize: 12,
                    decoration: TextDecoration.underline,
                  ),
                ),
              ),
            ),
            const SizedBox(height: 8),
            GestureDetector(
              onTap: _showBidDialog,
              child: Container(
                padding: const EdgeInsets.symmetric(
                  horizontal: 12,
```

```
                    vertical: 12,
                  ),
                  decoration: BoxDecoration(
                    color: const Color(0xFFFF4D4F),
                    borderRadius: BorderRadius.circular(10),
                  ),
                  child: const Column(
                    mainAxisSize: MainAxisSize.min,
                    children: [
                      Text(
                        '■■■',
                        style: TextStyle(color: Colors.white, fontSize: 10),
                      ),
                      Text(
                        '■■',
                        style: TextStyle(
                          color: Colors.white,
                          fontSize: 14,
                          fontWeight: FontWeight.bold,
                          decoration: TextDecoration.underline,
                        ),
                      ),
                    ],
                  ),
                ),
              ),
            ],
          ],
        ),
      );
    }

    Widget _buildAnnouncementBar() {
      // ■■■■■■■■■■■■■■
      if (_currentAnnounce == null) {
        return const SizedBox.shrink();
      }

      final content = _currentAnnounce!.content ?? '';
      final displayText = content.length > 18
          ? '${content.substring(0, 18)}...'
          : content;

      return GestureDetector(
        onTap: _onAnnouncementTap,
        child: Container(
          width: double.infinity,
          padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
          color: const Color(0xFFFF7144),
          child: Row(
```

```
          children: [
            const Icon(Icons.campaign, color: Colors.white, size: 16),
            const SizedBox(width: 8),
            Expanded(
              child: Text(
                displayText,
                style: const TextStyle(color: Colors.white, fontSize: 13),
                overflow: TextOverflow.ellipsis,
              ),
            ),
            const Icon(Icons.chevron_right, color: Colors.white, size: 16),
          ],
        ),
      ),
    );
  }

  /// ■■■■■■■■
  Future<void> _loadAnnouncement() async {
    try {
      final announce = await _announceRepository.getLatestAnnounce(
        categoryId: _announceCategoryId,
      );
      if (announce != null && mounted) {
        setState(() {
          _currentAnnounce = announce;
        });
        // ■■■■■■■■■■■
        await _checkAndShowAnnouncementDialog(announce);
      }
    } catch (e) {
      debugPrint('[AuctionChat] ■■■■■■■: $e');
    }
  }

  /// ■■■■■■■■■■■■■■■■■
  Future<void> _checkAndShowAnnouncementDialog(AnnounceDto announce) async {
    try {
      final prefs = await SharedPreferences.getInstance();
      final storedNotice = prefs.getString(_auctionNoticeKey);

      if (storedNotice == null || storedNotice.isEmpty) {
        // ■■■■■■■■■■■■
        if (mounted) {
          setState(() {
            _showAnnouncementDialog = true;
          });
        }
      } else {
        // ■■■■■■■■ ID ■■■■■■■■
```

```
        final storedData = jsonDecode(storedNotice) as Map<String, dynamic>;
        final storedId = storedData['id'];
        if (storedId != announce.id) {
          // ■■■■■■■■■■
          if (mounted) {
            setState(() {
              _showAnnouncementDialog = true;
            });
          }
        }
      }
    } catch (e) {
      debugPrint('[AuctionChat] ■■■■■■■■: $e');
    }
  }

  /// ■■■■■■■
  void _onAnnouncementTap() {
    // ■■■■■■■■
    // ■■ rootNavigator ■■■ go_router navigation shell ■■
    Navigator.of(context, rootNavigator: true).push(
      MaterialPageRoute(
        builder: (context) => AnnounceListPage(categoryId: _announceCategoryId),
      ),
    );
  }

  /// ■■■■■■■■■
  void _displayAnnouncementDialog() {
    if (_currentAnnounce == null || _isShowingAnnouncementDialog) return;

    _isShowingAnnouncementDialog = true; // ■■■■■■■■

    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (context) => AlertDialog(
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)),
        title: const Text(
          '■■',
          textAlign: TextAlign.center,
          style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
        ),
        content: SingleChildScrollView(
          child: Column(
            mainAxisSize: MainAxisSize.min,
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              // ■■■■
              if (_currentAnnounce!.imageUrl != null &&
```

```
                  _currentAnnounce!.imageUrl!.isNotEmpty)
                GestureDetector(
                  onTap: () => _previewImage(_currentAnnounce!.imageUrl!),
                  child: ClipRRect(
                    borderRadius: BorderRadius.circular(8),
                    child: CachedNetworkImage(
                      imageUrl: _currentAnnounce!.imageUrl!,
                      fit: BoxFit.contain,
                      height: 200,
                      placeholder: (_, __) =>
                          const Center(child: CircularProgressIndicator()),
                      errorWidget: (_, __, ___) => const Icon(
                        Icons.broken_image,
                        size: 64,
                        color: Colors.grey,
                      ),
                    ),
                  ),
                ),
              if (_currentAnnounce!.imageUrl != null &&
                  _currentAnnounce!.imageUrl!.isNotEmpty)
                const SizedBox(height: 16),
              // ■■■■
              Text(
                _currentAnnounce!.content ?? '',
                style: const TextStyle(fontSize: 14, height: 1.5),
              ),
            ],
          ),
        ),
        actions: [
          TextButton(
            onPressed: () {
              _closeAnnouncementDialog();
              Navigator.pop(context);
            },
            child: const Text('■■'),
          ),
        ],
      ),
    );
  }

  /// ■■■■■■■■■■■
  Future<void> _closeAnnouncementDialog() async {
    try {
      final prefs = await SharedPreferences.getInstance();
      final noticeData = jsonEncode(_currentAnnounce!.toJson());
      await prefs.setString(_auctionNoticeKey, noticeData);
      if (mounted) {
```

```
        setState(() {
          _showAnnouncementDialog = false;
        });
      }
    } catch (e) {
      debugPrint('[AuctionChat] ■■■■■■■■: $e');
    }
  }

  /// ■■■■
  void _previewImage(String url) {
    Navigator.push(
      context,
      MaterialPageRoute(
        builder: (context) =>
            _ImagePreviewPage(imageUrls: [url], initialIndex: 0),
      ),
    );
  }

  Widget _buildEmptyState() {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Icon(Icons.gavel, size: 64, color: Colors.grey.shade400),
          const SizedBox(height: 16),
          Text(
            '■■■■■■■■',
            style: TextStyle(fontSize: 16, color: Colors.grey.shade500),
          ),
        ],
      ),
    );
  }

  Widget _buildUnifiedMessageList(List<ChatMessage> messages) {
    return ListView.builder(
      controller: _scrollController, // ■■■■■■■■■■
      // ■■■■■■■■■■■56px■+ ■■■■■■■■■■■■■
      padding: const EdgeInsets.only(left: 16, right: 16, top: 16, bottom: 16),
      itemCount: messages.isEmpty ? 1 : messages.length,
      itemBuilder: (context, index) {
        if (messages.isEmpty) {
          return _buildEmptyState(); // ■■■■■■■■
        }
        final message = messages[index];
        return _buildMessageItem(message);
      },
    );
```

```
  }

  Widget _buildMessageItem(ChatMessage message) {
    final isSelf = message.from != null && message.from == _getCurrentUserId();

    if (message.type == ChatMessageType.welcome ||
        message.type == ChatMessageType.banUser ||
        message.type == ChatMessageType.backout) {
      return Padding(
        padding: const EdgeInsets.symmetric(vertical: 8),
        child: MessageWidget(message: message),
      );
    }

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 8),
      child: Row(
        mainAxisAlignment: isSelf
            ? MainAxisAlignment.end
            : MainAxisAlignment.start,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          if (!isSelf) ...[
            _buildAvatar(message, onTap: () => _onAvatarTap(message)),
            const SizedBox(width: 10),
          ],
          Flexible(
            child: Column(
              crossAxisAlignment: isSelf
                  ? CrossAxisAlignment.end
                  : CrossAxisAlignment.start,
              children: [
                if (!isSelf) _buildUserName(message),
                const SizedBox(height: 4),
                MessageWidget(
                  message: message,
                  onTap: () => _onMessageTap(message),
                  onLongPress: () => _onMessageLongPress(message),
                ),
              ],
            ),
          ),
          if (isSelf) ...[
            const SizedBox(width: 10),
            _buildAvatar(message, onTap: () => _onAvatarTap(message)),
          ],
        ],
      ),
    );
  }
```

```
1101
1102    /// ■■■■■■■■
1103    void _onMessageTap(ChatMessage message) {
1104      // ■■■■■■■■■■■■■■■■
1105      if (message.type == ChatMessageType.auctionStart ||
1106          message.type == ChatMessageType.auctionBid ||
1107          message.type == ChatMessageType.auctionEnd ||
1108          message.type == ChatMessageType.auctionDeal) {
1109        _showAuctionDetailFromMessage(message);
1110      }
1111    }
1112
1113    /// ■■■■■■■■
1114    void _onAvatarTap(ChatMessage message) {
1115      _showMessageActionSheet(message, isAvatarTap: true);
1116    }
1117
1118    /// ■■■■■■■■
1119    void _onMessageLongPress(ChatMessage message) {
1120      _showMessageActionSheet(message, isAvatarTap: false);
1121    }
1122
1123    /// ■■■■■■■■
1124    void _showMessageActionSheet(
1125      ChatMessage message, {
1126      bool isAvatarTap = false,
1127    }) {
1128      final currentUserId = _getCurrentUserId();
1129      final isSelf = message.from == currentUserId;
1130      final isImage = message.type == ChatMessageType.image;
1131      final isAdmin = ref.read(userProvider.notifier).isAdmin;
1132      final senderIsAdmin = message.fromAdmin ?? false;
1133
1134      // ■■■■
1135      FocusScope.of(context).unfocus();
1136
1137      final actions = <ActionSheetItem>[];
1138
1139      // ■■■■■■■■■■■■ UniApp ■■■■■■■■■■■■■■■
1140      if (isImage) {
1141        actions.add(ActionSheetItem(
1142          icon: Icons.favorite_border,
1143          title: '■■■■■■',
1144          onTap: () => _addToFavorites(message),
1145        ));
1146      }
1147
1148      // ■■■■■■■■■■■
1149      if (isSelf) {
1150        actions.add(ActionSheetItem(
```

```
          icon: Icons.undo,
          title: '■■',
          onTap: () => _backoutMessage(message),
        ));
      }

      // ■■■■■■■■■■■■■■
      if (!isSelf) {
        actions.add(ActionSheetItem(
          icon: Icons.person_add,
          title: '■■■■',
          onTap: () => _addFriend(message),
        ));
      }

      // ■■■■■■■■■■■■■■■■■■■■■■
      if ((isAdmin || senderIsAdmin) && !isSelf) {
        actions.add(ActionSheetItem(
          icon: Icons.chat_bubble_outline,
          title: '■■',
          onTap: () => _startPrivateChat(message),
        ));
      }

      // ■■■■■■■■■■■■■■
      if (!isSelf) {
        actions.add(ActionSheetItem(
          icon: Icons.person,
          title: '■■■■',
          onTap: () => _viewUserInfo(message),
        ));
      }

      showAppBottomSheet(
        context: context,
        builder: (context) => AppActionSheet(
          actions: actions,
          onCancel: () {},
        ),
      );
    }

    /// ■■■■ SnackBar ■■
    void _showTopSnackBar(
      BuildContext context,
      String message, {
      bool isError = false,
    }) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
```

```
        content: Text(message, style: const TextStyle(color: Colors.white)),
        backgroundColor: isError
            ? const Color(0xFFF44336)
            : const Color(0xFF4CAF50),
        behavior: SnackBarBehavior.floating,
        shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(8)),
        margin: EdgeInsets.only(
          left: 16,
          right: 16,
          top: MediaQuery.of(context).padding.top + 50,
          bottom: MediaQuery.of(context).padding.bottom + 80,
        ),
        duration: Duration(seconds: isError ? 3 : 2),
      ),
    );
  }

  /// ■■■■
  Future<void> _addFriend(ChatMessage message) async {
    if (message.from == null) return;

    try {
      final friendRepository = FriendRepository();
      await friendRepository.addFriend(message.from!);
      if (mounted) {
        _showTopSnackBar(context, '■■■■■■■');
      }
    } catch (e) {
      if (mounted) {
        final errorText = e.toString().replaceAll('Exception: ', '');
        _showTopSnackBar(context, errorText, isError: true);
      }
    }
  }

  /// ■■■■
  void _startPrivateChat(ChatMessage message) {
    if (message.from == null) return;
    final friendId = message.from!;
    final friendName = message.fromName ?? '■■';
    final friendAvatar = message.avatar;
    context.push(
      '/chat/private/$friendId?name=$friendName${friendAvatar != null ? '&avatar=$friendAvatar' : ''
    );
  }

  /// ■■■■■■
  Future<void> _viewUserInfo(ChatMessage message) async {
    if (message.from == null) return;

```

```
    // ■■■■■
    showDialog(
      context: context,
      barrierDismissible: false,
      builder: (context) => const Center(child: CircularProgressIndicator()),
    );

    try {
      final userRepository = UserRepository();
      final user = await userRepository.getUserById(message.from!);

      if (!mounted) return;

      // ■■■■■■
      Navigator.pop(context);

      if (user != null) {
        UserProfileDialog.show(context, user);
      } else {
        _showTopSnackBar(context, '■■■■■■■■', isError: true);
      }
    } catch (e) {
      if (!mounted) return;
      Navigator.pop(context);
      final errorText = e.toString().replaceAll('Exception: ', '');
      _showTopSnackBar(context, errorText, isError: true);
    }
  }

  /// ■■■■
  Future<void> _backoutMessage(ChatMessage message) async {
    if (message.id == null) return;

    try {
      final chatRepository = ChatRepository();
      await chatRepository.backoutMessage(message.id!);
      // ■■■■■■■■■
      ref.read(chatStoreProvider.notifier).removeMessage(message.id!);
      if (mounted) {
        _showTopSnackBar(context, '■■■');
      }
    } catch (e) {
      if (mounted) {
        final errorText = e.toString().replaceAll('Exception: ', '');
        _showTopSnackBar(context, errorText, isError: true);
      }
    }
  }

  /// ■■■■■■■■■■■
```

```
  Future<void> _addToFavorites(ChatMessage message) async {
    final payload = message.payload;
    if (payload == null) return;

    // ■■■■ URL
    String? imageUrl;
    if (payload is Map<String, dynamic>) {
      imageUrl = payload['url'] as String?;
    } else if (payload is String) {
      imageUrl = payload;
    }

    if (imageUrl == null || imageUrl.isEmpty) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■')));
      }
      return;
    }

    try {
      // ■■■■■■ API
      final success = await ref
          .read(userEmojiProvider.notifier)
          .addToEmoji(imageUrl);

      if (mounted) {
        if (success) {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■')));
        } else {
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■')));
        }
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text('■■■■: $e')));
      }
    }
  }

  /// ■■■■■■■■■■■■■■■■
  void _onSelectFavoriteEmoji(dynamic emoji) {
    final url = emoji.url;
```

```
    if (url == null || url.isEmpty) return;

    // ■■ payload■■ UniApp ■■■■
    final payload = {'url': url, 'width': 200, 'height': 200};

    // ■■■■■■
    ref
        .read(chatStoreProvider.notifier)
        .sendChannelMsg(
          channel: _channel,
          message: url,
          type: ChatMessageType.image,
          payload: payload,
        );

    // ■■■■■
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _scrollToBottom();
    });
  }

  /// ■■■■■■■■■■■■■■■■
  void _showAuctionDetailFromMessage(ChatMessage message) {
    final payload = message.payload;
    if (payload == null) return;

    // payload ■■■ ChatMessage.fromJson ■■■■ Map
    if (payload is Map<String, dynamic>) {
      // ■■ AuctionItemDto.fromJson ■■■■■■
      final item = AuctionItemDto.fromJson(payload);
      _showAuctionDetail(item);
    }
  }

  int? _getCurrentUserId() {
    final userState = ref.read(userProvider);
    return userState.user?.id;
  }

  Widget _buildAvatar(ChatMessage message, {VoidCallback? onTap}) {
    // ■■■■■■■■ avatar ■■
    String? avatarUrl;
    if (message.avatar != null) {
      final avatar = message.avatar!;
      avatarUrl = avatar.startsWith('http')
          ? avatar
          : 'https://image.molitao.top/$avatar';
    }

    Widget avatarWidget;
```

```
    if (avatarUrl == null || avatarUrl.isEmpty) {
      // ■■■■■■■■■■ + ■■■
      avatarWidget = Container(
        width: 36,
        height: 36,
        decoration: BoxDecoration(
          color: _getAvatarColor(message.from ?? 0),
          shape: BoxShape.circle,
        ),
        child: Center(
          child: Text(
            _getAvatarText(message.fromName ?? '■■'),
            style: const TextStyle(
              color: Colors.white,
              fontSize: 12,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
      );
    } else {
      // ■■■■■■
      avatarWidget = Container(
        width: 36,
        height: 36,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          image: DecorationImage(
            image: NetworkImage(avatarUrl),
            fit: BoxFit.cover,
          ),
        ),
      );
    }

    return GestureDetector(onTap: onTap, child: avatarWidget);
  }

  Widget _buildUserName(ChatMessage message) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        if (message.fromAdmin == true) ...[
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
            decoration: BoxDecoration(
              color: Colors.red,
              borderRadius: BorderRadius.circular(4),
            ),
```

```
              child: const Text(
                '■■',
                style: TextStyle(
                  color: Colors.white,
                  fontSize: 10,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ),
            const SizedBox(width: 6),
          ] else if (message.userChatLevel != null) ...[
            // ■■■■■
            Container(
              padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
              decoration: BoxDecoration(
                color: _getLevelBackgroundColor(message.userChatLevel!),
                borderRadius: BorderRadius.circular(4),
              ),
              child: Text(
                message.userChatLevel!['name'] ?? '',
                style: TextStyle(
                  color: _getLevelTextColor(message.userChatLevel!),
                  fontSize: 10,
                  fontWeight: FontWeight.bold,
                ),
              ),
            ),
            const SizedBox(width: 6),
          ],
          Text(
            message.fromName ?? '■■■■',
            style: const TextStyle(fontSize: 12, color: Color(0xFF999999)),
          ),
        ],
      );
    }

    Color _getLevelBackgroundColor(Map<String, dynamic> level) {
      final levelNum = level['level'] as int? ?? 0;
      if (levelNum == 7) {
        return Colors.black; // ■7■■■■■■
      }
      if (levelNum == 8) {
        return const Color(0xFF000000); // ■8■■■■■■
      }
      // ■■■■■■■■■
      final borderColor = level['borderColor'] as String? ?? '#F4835A';
      return _parseColor(borderColor);
    }

```

```
  Color _getLevelTextColor(Map<String, dynamic> level) {
    final levelNum = level['level'] as int? ?? 0;
    if (levelNum == 7 || levelNum == 8) {
      return const Color(0xFFFFD700); // ■■
    }
    return Colors.white;
  }

  Color _parseColor(String colorStr) {
    try {
      if (colorStr.startsWith('#') && colorStr.length >= 7) {
        final hex = colorStr.replaceFirst('#', '');
        return Color(int.parse('FF$hex', radix: 16));
      }
    } catch (_) {}
    return const Color(0xFFF4835A); // ■■■■■
  }

  void _showAuctionDetail(AuctionItemDto item) {
    // ■■ description ■■■■ URL ■■■■■■■■
    final imageUrls = _extractImageUrls(item.description);

    // ■■■■■■■■■ modal ■■■■■
    FocusScope.of(context).unfocus();

    showScrollableBottomSheet(
      context: context,
      builder: (context) => DraggableScrollableSheet(
        initialChildSize: 0.6,
        minChildSize: 0.3,
        maxChildSize: 0.9,
        expand: false,
        builder: (context, scrollController) => SingleChildScrollView(
          controller: scrollController,
          child: Container(
            padding: const EdgeInsets.all(16),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              mainAxisSize: MainAxisSize.min,
              children: [
                // ■■■
                Row(
                  mainAxisAlignment: MainAxisAlignment.spaceBetween,
                  children: [
                    Expanded(
                      child: Text(
                        item.name ?? '■■■■',
                        style: const TextStyle(
                          fontSize: 18,
                          fontWeight: FontWeight.bold,
```

```
                          color: Color(0xFFFF7144),
                        ),
                      ),
                    ),
                    IconButton(
                      icon: const Icon(Icons.close),
                      onPressed: () => Navigator.pop(context),
                    ),
                  ],
                ),
                const SizedBox(height: 16),
                // ■■■■■■■■■■■■■
                if (item.status == AuctionStatusEnum.listed)
                  Padding(
                    padding: const EdgeInsets.only(bottom: 12),
                    child: ElevatedButton(
                      onPressed: () => _subscribeNotification(item.id),
                      style: ElevatedButton.styleFrom(
                        backgroundColor: const Color(0xFF4CAF50),
                        foregroundColor: Colors.white,
                        padding: const EdgeInsets.symmetric(
                          horizontal: 16,
                          vertical: 8,
                        ),
                        shape: RoundedRectangleBorder(
                          borderRadius: BorderRadius.circular(8),
                        ),
                      ),
                      child: const Text('■■■■'),
                    ),
                  ),
                // ■■■■■■
                _buildAuctionContent(item, imageUrls),
              ],
            ),
          ),
        ),
      ),
    );
  }

  /// ■ description HTML ■■■■■ URL ■■
  List<String> _extractImageUrls(String? description) {
    if (description == null || description.isEmpty) return [];

    final urls = <String>[];

    // ■■ data-url ■■
    final dataUrlRegex = RegExp(
      r'<img[^>]+data-url=["\x27]([^"\x27]+)["\x27]',
```

```
    );
    final dataUrlMatches = dataUrlRegex.allMatches(description);
    for (final match in dataUrlMatches) {
      urls.add(match.group(1)!);
    }

    // ■■■■ data-url■■■ src ■■
    if (urls.isEmpty) {
      final srcRegex = RegExp(r'<img[^>]+src=["\x27"]([^"\x27"]+)["\x27"]');
      final srcMatches = srcRegex.allMatches(description);
      for (final match in srcMatches) {
        final url = match.group(1)!;
        // ■■■■■■■ !w300
        final cleanUrl = url.replaceAll(RegExp(r'!w300$'), '');
        urls.add(cleanUrl);
      }
    }

    return urls;
  }

  /// ■■■■ URL ■■■■■
  String? _convertImageUrl(String? url) {
    if (url == null || url.isEmpty) return null;

    // ■■ URL
    url = url.trim();

    // ■■ file:// ■■■■■■■■■■■■■
    if (url.startsWith('file://')) {
      // ■■■■■■■■■■■■■■
      url = url.replaceFirst('file://', '');
      if (!url.startsWith('/')) {
        url = '/$url';
      }
    }

    // ■■■■■■■■ / ■■■
    if (url.startsWith('/')) {
      return 'https://image.molitao.top$url';
    } else if (!url.startsWith('http://') && !url.startsWith('https://')) {
      // ■■■■■■
      return 'https://image.molitao.top/$url';
    }

    return url;
  }

  /// ■■■■■■■■■ UniApp getStartContent■
  Widget _buildAuctionContent(AuctionItemDto item, List<String> imageUrls) {
```

```
    final description = item.description;

    // ■■■■ description■■■ imageUrl
    if (description == null || description.trim().isEmpty) {
      final imageUrl = _convertImageUrl(item.imageUrl);
      if (imageUrl != null && imageUrl.isNotEmpty) {
        return GestureDetector(
          onTap: () => _previewImages([imageUrl]),
          child: ClipRRect(
            borderRadius: BorderRadius.circular(8),
            child: CachedNetworkImage(
              imageUrl: imageUrl,
              width: double.infinity,
              height: 200,
              fit: BoxFit.cover,
              placeholder: (_, __) => Container(
                height: 200,
                color: Colors.grey.shade200,
                child: const Center(
                  child: SizedBox(
                    width: 24,
                    height: 24,
                    child: CircularProgressIndicator(strokeWidth: 2),
                  ),
                ),
              ),
              errorWidget: (_, __, ___) => Container(
                height: 200,
                color: Colors.grey.shade200,
                child: const Center(
                  child: Icon(Icons.image, size: 48, color: Colors.grey),
                ),
              ),
            ),
          ),
        );
      }
      return const SizedBox.shrink();
    }

    // ■■ description HTML
    return GestureDetector(
      onTap: () {
        // ■■■■■■■■■■
        if (imageUrls.isNotEmpty) {
          _previewImages(imageUrls);
        }
      },
      child: Html(
        data: description,
```

```
          style: {
            "body": Style(margin: Margins.zero, padding: HtmlPaddings.zero),
            "img": Style(width: Width(double.infinity)),
            "div": Style(fontSize: FontSize(14), color: Colors.black87),
            "span": Style(fontSize: FontSize(14), color: Colors.black87),
          },
          extensions: [
            TagExtension(
              tagsToExtend: {"img"},
              builder: (extensionContext) {
                // ■■■■ data-url■■■■■ src
                final dataUrl = extensionContext.attributes['data-url'];
                final src = extensionContext.attributes['src'];
                String? imageUrl = dataUrl ?? src;

                if (imageUrl == null) return const SizedBox.shrink();

                // ■■ URL
                imageUrl = imageUrl.trim();

                // ■■■■■■■ !w300
                imageUrl = imageUrl.replaceAll(RegExp(r'!w300$'), '');

                // ■■ file:// ■■■■■■■■■■■■■
                if (imageUrl.startsWith('file://')) {
                  // ■■■■■■■■■■■■■■■
                  imageUrl = imageUrl.replaceFirst('file://', '');
                  if (!imageUrl.startsWith('/')) {
                    imageUrl = '/$imageUrl';
                  }
                }

                // ■■■■■■■■ / ■■■
                if (imageUrl.startsWith('/')) {
                  imageUrl = 'https://image.molitao.top$imageUrl';
                } else if (!imageUrl.startsWith('http://') &&
                    !imageUrl.startsWith('https://')) {
                  // ■■■■■■
                  imageUrl = 'https://image.molitao.top/$imageUrl';
                }

                return GestureDetector(
                  onTap: () {
                    if (imageUrls.isNotEmpty) {
                      _previewImages(imageUrls);
                    } else if (imageUrl != null) {
                      _previewImages([imageUrl]);
                    }
                  },
                  child: ClipRRect(
```

```
                    borderRadius: BorderRadius.circular(4),
                    child: CachedNetworkImage(
                      imageUrl: imageUrl,
                      width: double.infinity,
                      fit: BoxFit.cover,
                      placeholder: (_, __) => Container(
                        height: 150,
                        color: Colors.grey.shade200,
                        child: const Center(
                          child: SizedBox(
                            width: 24,
                            height: 24,
                            child: CircularProgressIndicator(strokeWidth: 2),
                          ),
                        ),
                      ),
                      errorWidget: (_, __, ___) => const SizedBox.shrink(),
                    ),
                  ),
                );
              },
            ),
          ],
        ),
      );
    }

    /// ■■■■
    void _previewImages(List<String> urls) {
      if (urls.isEmpty) return;

      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) =>
              _ImagePreviewPage(imageUrls: urls, initialIndex: 0),
        ),
      );
    }

    /// ■■■■■■
    Widget _buildStatusBadge(AuctionStatusEnum? status) {
      String text;
      Color color;

      switch (status) {
        case AuctionStatusEnum.auctioning:
          text = '■■■';
          color = const Color(0xFF4CAF50);
          break;
```

```
      case AuctionStatusEnum.listed:
        text = '■■■';
        color = const Color(0xFF999999);
        break;
      case AuctionStatusEnum.sold:
        text = '■■■';
        color = const Color(0xFF4CAF50);
        break;
      default:
        text = '■■■■';
        color = const Color(0xFF999999);
    }

    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 6),
      decoration: BoxDecoration(
        color: color,
        borderRadius: BorderRadius.circular(4),
      ),
      child: Text(
        text,
        style: const TextStyle(color: Colors.white, fontSize: 12),
      ),
    );
  }

  /// ■■■■■■
  /// ■■■■■■■■■■■ + ■■■■
  Future<void> _subscribeNotification(int? auctionItemId) async {
    debugPrint('[■■■■] ■■■■, auctionItemId=$auctionItemId');
    if (auctionItemId == null) {
      debugPrint('[■■■■] auctionItemId ■■■■■');
      return;
    }

    // 1. ■■ APP ■■■■
    final permissionService = NotificationPermissionService();
    final hasPermission = await permissionService.checkPermission();
    if (!hasPermission) {
      debugPrint('[■■■■] ■■■■■■■■■■■');
      await permissionService.showPermissionDialog(context);
      return;
    }

    // 2. ■■■■■■ OpenId
    final authRepository = AuthRepository();
    final openId = await authRepository.getMyWechatOpenId();
    debugPrint('[■■■■] ■■ OpenId: $openId');

    // 3. ■■■ OpenId■■■■■■■■■■■■■■■
```

```
    String? subscribedOpenId;
    if (openId != null && openId.isNotEmpty) {
      try {
        final weChatService = WeChatService();
        // ■■■■■■■■■■■■■■■■■■■■■
        // ■■■■■■ subscribed=true■■■■■■■■■■ subscribed=false
        final subscribed = await weChatService.requestSubscribeMessageWithCallback(
          scene: 1,
          reserved: auctionItemId.toString(),
        );
        debugPrint('[■■■■■] ■■■■■■■■■■■: $subscribed');
        if (subscribed) {
          subscribedOpenId = openId;
        }
      } catch (e) {
        debugPrint('[■■■■■] ■■■■■■■: $e');
      }
    } else {
      debugPrint('[■■■■■] ■■■■■ OpenId■■■■■■■■');
    }

    // 4. ■■■■■■■■■■■
    // ■■■■■■ platform ■ openid ■■■■■■■■■■■■■■ + ■■■■■
    await _saveSubscription(auctionItemId, openid: subscribedOpenId);
  }

  Future<void> _saveSubscription(int auctionItemId, {String? openid}) async {
    debugPrint('[■■] ■■■■■■■■■, auctionItemId=$auctionItemId, openid=$openid');
    try {
      final success = await ref
          .read(auctionProvider.notifier)
          .subscribeStartNotification(auctionItemId, openid: openid);
      debugPrint('[■■] ■■■■■■■: success=$success');

      if (mounted) {
        Navigator.pop(context);
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
            content: Text(success ? '■■■■■■■■■■■■■■■■■' : '■■■■■■■■■'),
            backgroundColor: success ? const Color(0xFF4CAF50) : Colors.red,
          ),
        );
      }
    } catch (e, stack) {
      debugPrint('[■■] ■■■■■■■: $e');
      debugPrint('[■■] ■■: $stack');
      if (mounted) {
        Navigator.pop(context);
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(
```

```
            content: Text('■■■■: $e'),
            backgroundColor: Colors.red,
          ),
        );
      }
    }
  }

  Color _getAvatarColor(int userId) {
    final colors = [
      const Color(0xFFF4835A),
      const Color(0xFF1890FF),
      const Color(0xFFFF4D4F),
      const Color(0xFF722ED1),
      const Color(0xFF52C41A),
      const Color(0xFFFA8C16),
    ];
    return colors[userId % colors.length];
  }

  String _getAvatarText(String name) {
    if (name.isEmpty) return '■';
    return name.length > 2 ? name.substring(0, 2) : name;
  }

  Widget _buildNewMessageButton(int unreadCount) {
    return Positioned(
      right: 60,
      top: 60,
      child: SlideTransition(
        position: _unreadAnimation,
        child: GestureDetector(
          onTap: _scrollToBottom,
          child: Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, vertical: 8),
            decoration: const BoxDecoration(
              color: Color(0xFFF4835A),
              borderRadius: BorderRadius.only(
                topLeft: Radius.circular(8),
                bottomLeft: Radius.circular(8),
              ),
            ),
            child: Text(
              '$unreadCount■■■■',
              style: const TextStyle(color: Colors.white, fontSize: 12),
            ),
          ),
        ),
      ),
    );
```

```
  }

  Widget _buildAuctionListPanel(dynamic auctionState) {
    return Positioned.fill(
      child: GestureDetector(
        onTap: _toggleAuctionList,
        child: Container(
          color: Colors.black54,
          child: Align(
            alignment: Alignment.centerRight,
            child: GestureDetector(
              onTap: () {},
              child: SlideTransition(
                position: _auctionListAnimation,
                child: Container(
                  width: 280,
                  height: double.infinity,
                  color: Colors.white,
                  child: Column(
                    children: [
                      // Tab header
                      Container(
                        height: 48,
                        margin: const EdgeInsets.all(8),
                        decoration: BoxDecoration(
                          color: Colors.white,
                          borderRadius: BorderRadius.circular(8),
                          border: Border.all(color: Colors.grey.shade300),
                        ),
                        child: Row(
                          children: [
                            Expanded(
                              child: GestureDetector(
                                onTap: () {
                                  ref
                                      .read(auctionProvider.notifier)
                                      .setActiveAuctionTab(1);
                                },
                                child: Container(
                                  decoration: BoxDecoration(
                                    color: auctionState.activeAuctionTab == 1
                                        ? const Color(
                                            0xFFF4835A,
                                          ) // Active tab background
                                        : Colors
                                              .white, // Inactive tab background
                                    borderRadius: BorderRadius.horizontal(
                                      left: Radius.circular(8),
                                    ),
                                    border: Border.all(
```

```
                                color: Colors.grey.shade300,
                                width: auctionState.activeAuctionTab == 1
                                    ? 0
                                    : 1,
                              ),
                            ),
                            child: Center(
                              child: Text(
                                '■■■■',
                                style: TextStyle(
                                  color:
                                      auctionState.activeAuctionTab == 1
                                      ? Colors
                                            .white // Active tab text
                                      : const Color(
                                          0xFF666666,
                                        ), // Inactive tab text
                                  fontWeight: FontWeight.bold,
                                ),
                              ),
                            ),
                          ),
                        ),
                      ),
                    ),
                  ),
                  Expanded(
                    child: GestureDetector(
                      onTap: () {
                        ref
                            .read(auctionProvider.notifier)
                            .setActiveAuctionTab(2);
                      },
                      child: Container(
                        decoration: BoxDecoration(
                          color: auctionState.activeAuctionTab == 2
                              ? const Color(
                                  0xFFF4835A,
                                ) // Active tab background
                              : Colors
                                    .white, // Inactive tab background
                          borderRadius: BorderRadius.horizontal(
                            right: Radius.circular(8),
                          ),
                          border: Border.all(
                            color: Colors.grey.shade300,
                            width: auctionState.activeAuctionTab == 2
                                ? 0
                                : 1,
                          ),
                        ),
                        child: Center(
```

```
                                child: Text(
                                  '■■■■',
                                  style: TextStyle(
                                    color:
                                        auctionState.activeAuctionTab == 2
                                        ? Colors
                                              .white // Active tab text
                                        : const Color(
                                            0xFF666666,
                                          ), // Inactive tab text
                                    fontWeight: FontWeight.bold,
                                  ),
                                ),
                              ),
                            ),
                          ),
                        ),
                      ),
                    ],
                  ),
                ),
                // Tab content area
                Expanded(
                  child: (() {
                    if (auctionState.activeAuctionTab == 1) {
                      // Tab 1: ■■■■ (Today's list - listed and auctioning items)
                      final todayList = auctionState.todayList;

                      if (auctionState.isLoading && todayList.isEmpty) {
                        return const Center(
                          child: CircularProgressIndicator(),
                        );
                      }

                      if (todayList.isEmpty) {
                        return const Center(child: Text('■■■■'));
                      }

                      return ListView.builder(
                        padding: const EdgeInsets.all(8),
                        itemCount: todayList.length,
                        itemBuilder: (context, index) {
                          final item = todayList[index];
                          final isAuctioning =
                              item.status == AuctionStatusEnum.auctioning;

                          return Container(
                            margin: const EdgeInsets.symmetric(
                              vertical: 4,
                            ),
                            decoration: BoxDecoration(
```

```
                color: const Color(0xFFF5F5F5),
                borderRadius: BorderRadius.circular(8),
              ),
              child: ListTile(
                leading: ClipRRect(
                  borderRadius: BorderRadius.circular(4),
                  child:
                      item.imageUrl != null &&
                          item.imageUrl!.isNotEmpty
                      ? CachedNetworkImage(
                          imageUrl: item.imageUrl!,
                          width: 50,
                          height: 50,
                          fit: BoxFit.cover,
                          placeholder: (_, __) => Container(
                            width: 50,
                            height: 50,
                            color: Colors.grey.shade200,
                            child: const SizedBox(
                              width: 16,
                              height: 16,
                              child:
                                  CircularProgressIndicator(
                                    strokeWidth: 2,
                                  ),
                            ),
                          ),
                          errorWidget: (_, __, ___) =>
                              Container(
                                width: 50,
                                height: 50,
                                color: Colors.grey.shade200,
                                child: const Icon(
                                  Icons.image,
                                  size: 24,
                                ),
                              ),
                        )
                      : Container(
                          width: 50,
                          height: 50,
                          color: Colors.grey.shade200,
                          child: const Icon(
                            Icons.image,
                            size: 24,
                          ),
                        ),
                ),
                title: Text(
                  item.name ?? '■■■■',
```

```
                                maxLines: 1,
                                overflow: TextOverflow.ellipsis,
                              ),
                              subtitle: item.currentPrice != null
                                  ? Text('¥${item.currentPrice}')
                                  : null,
                              trailing: Container(
                                padding: const EdgeInsets.symmetric(
                                  horizontal: 8,
                                  vertical: 4,
                                ),
                                decoration: BoxDecoration(
                                  color: isAuctioning
                                      ? const Color(0xFF4CAF50)
                                      : Colors.grey,
                                  borderRadius: BorderRadius.circular(4),
                                ),
                                child: Text(
                                  isAuctioning ? '■■■' : '■■',
                                  style: const TextStyle(
                                    color: Colors.white,
                                    fontSize: 12,
                                  ),
                                ),
                              ),
                              onTap: () => _showAuctionDetail(item),
                            ),
                          );
                        },
                      );
                    } else {
                      // Tab 2: ■■■■ (Yesterday's list - sold items)
                      if (auctionState.isLoading &&
                          auctionState.yesterdayList.isEmpty) {
                        return const Center(
                          child: CircularProgressIndicator(),
                        );
                      }

                      if (auctionState.yesterdayList.isEmpty) {
                        return const Center(child: Text('■■■■'));
                      }

                      return ListView.builder(
                        padding: const EdgeInsets.all(8),
                        itemCount: auctionState.yesterdayList.length,
                        itemBuilder: (context, index) {
                          final item = auctionState.yesterdayList[index];

                          return Container(
```

```
                                margin: const EdgeInsets.symmetric(
                                  vertical: 4,
                                ),
                                decoration: BoxDecoration(
                                  color: const Color(0xFFF5F5F5),
                                  borderRadius: BorderRadius.circular(8),
                                ),
                                child: ListTile(
                                  leading: ClipRRect(
                                    borderRadius: BorderRadius.circular(4),
                                    child:
                                        item.imageUrl != null &&
                                            item.imageUrl!.isNotEmpty
                                        ? CachedNetworkImage(
                                            imageUrl: item.imageUrl!,
                                            width: 50,
                                            height: 50,
                                            fit: BoxFit.cover,
                                            placeholder: (_, __) => Container(
                                              width: 50,
                                              height: 50,
                                              color: Colors.grey.shade200,
                                              child: const SizedBox(
                                                width: 16,
                                                height: 16,
                                                child:
                                                    CircularProgressIndicator(
                                                      strokeWidth: 2,
                                                    ),
                                              ),
                                            ),
                                            errorWidget: (_, __, ___) =>
                                                Container(
                                                  width: 50,
                                                  height: 50,
                                                  color: Colors.grey.shade200,
                                                  child: const Icon(
                                                    Icons.image,
                                                    size: 24,
                                                  ),
                                                ),
                                          )
                                        : Container(
                                            width: 50,
                                            height: 50,
                                            color: Colors.grey.shade200,
                                            child: const Icon(
                                              Icons.image,
                                              size: 24,
                                            ),
```

```
                                        ),
                                    ),
                                    title: Text(
                                      item.name ?? '■■■■',
                                      maxLines: 1,
                                      overflow: TextOverflow.ellipsis,
                                    ),
                                    subtitle:
                                        (item.finalPrice ??
                                                item.currentPrice) !=
                                            null
                                        ? Text(
                                            '¥${item.finalPrice ?? item.currentPrice}',
                                          )
                                        : null,
                                    trailing: Container(
                                      padding: const EdgeInsets.symmetric(
                                        horizontal: 8,
                                        vertical: 4,
                                      ),
                                      decoration: BoxDecoration(
                                        color: const Color(0xFF4CAF50),
                                        borderRadius: BorderRadius.circular(4),
                                      ),
                                      child: Text(
                                        '■■■',
                                        style: const TextStyle(
                                          color: Colors.white,
                                          fontSize: 12,
                                        ),
                                      ),
                                    ),
                                    onTap: () => _showAuctionDetail(item),
                                  ),
                                );
                              },
                            );
                          }
                        })(),
                      ),
                    ],
                  ),
                ),
              ),
            ),
          ),
        ),
      );
    }
```

```
}

/// ■■■■■■
class _ImagePreviewPage extends StatefulWidget {
  final List<String> imageUrls;
  final int initialIndex;

  const _ImagePreviewPage({required this.imageUrls, this.initialIndex = 0});

  @override
  State<_ImagePreviewPage> createState() => _ImagePreviewPageState();
}

class _ImagePreviewPageState extends State<_ImagePreviewPage> {
  late PageController _pageController;
  late int _currentIndex;

  @override
  void initState() {
    super.initState();
    _currentIndex = widget.initialIndex;
    _pageController = PageController(initialPage: widget.initialIndex);
  }

  @override
  void dispose() {
    _pageController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.black,
      appBar: AppBar(
        backgroundColor: Colors.black,
        foregroundColor: Colors.white,
        title: Text(
          '${_currentIndex + 1} / ${widget.imageUrls.length}',
          style: const TextStyle(fontSize: 20, color: Colors.white),
        ),
      ),
      body: PageView.builder(
        controller: _pageController,
        itemCount: widget.imageUrls.length,
        onPageChanged: (index) {
          setState(() {
            _currentIndex = index;
          });
        },
```

```
        itemBuilder: (context, index) {
          return InteractiveViewer(
            child: Center(
              child: CachedNetworkImage(
                imageUrl: widget.imageUrls[index],
                fit: BoxFit.contain,
                placeholder: (_, __) => const Center(
                  child: CircularProgressIndicator(
                    color: Colors.white,
                    strokeWidth: 2,
                  ),
                ),
                errorWidget: (_, __, ___) => const Center(
                  child: Icon(
                    Icons.broken_image,
                    color: Colors.white,
                    size: 64,
                  ),
                ),
              ),
            ),
          );
        },
      ),
    );
  }
}

/* === lib/presentation/pages/profile/profile_page.dart === */
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
import 'package:package_info_plus/package_info_plus.dart';
import '../../../domain/entities/my_count_entity.dart';
import '../../../data/api/api_client.dart';
import '../../../data/api/api_endpoints.dart';
import '../../../data/repositories/payment_repository.dart';
import '../../providers/user_provider.dart';
import '../../../core/widgets/app_bottom_sheet.dart';

/// ■■■■■■■■■■
const double _depositAmount = 51;

class ProfilePage extends ConsumerStatefulWidget {
  const ProfilePage({super.key});

  @override
  ConsumerState<ProfilePage> createState() => _ProfilePageState();
}
```

```
class _ProfilePageState extends ConsumerState<ProfilePage> {
  MyCountEntity _myCount = MyCountEntity(friend: 0, depositBalance: 0);
  bool _isLoadingCount = false;
  String _appVersion = '';

  @override
  void initState() {
    super.initState();
    _loadMyCount();
    _loadAppVersion();
  }

  Future<void> _loadAppVersion() async {
    try {
      final packageInfo = await PackageInfo.fromPlatform();
      if (mounted) {
        setState(() {
          _appVersion = packageInfo.version;
        });
      }
    } catch (e) {
      debugPrint('Failed to get package info: $e');
    }
  }

  /// ■■■■■■■■■■■■■■
  Future<void> _loadMyCount() async {
    final userState = ref.read(userProvider);
    if (!userState.isLoggedIn) {
      print('[ProfilePage] ■■■■■■■■■■■■■■■');
      return;
    }

    if (_isLoadingCount) return; // ■■■■■■

    setState(() => _isLoadingCount = true);

    try {
      print('[ProfilePage] ■■■■■■■■■...');
      final response = await ApiClient().dio.get(ApiEndpoints.getMyCount);
      print('[ProfilePage] API ■■: ${response.data}');

      if (response.data != null && mounted) {
        final entity = MyCountEntity.fromJson(
          response.data as Map<String, dynamic>,
        );
        print(
          '[ProfilePage] ■■■■: friend=${entity.friend}, depositBalance=${entity.depositBalance}',
        );
        setState(() {
```

```
          _myCount = entity;
          _isLoadingCount = false;
        });
      }
    } catch (e) {
      print('[ProfilePage] ■■■■■■■■: $e');
      if (mounted) {
        setState(() => _isLoadingCount = false);
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    final userState = ref.watch(userProvider);

    return Scaffold(
      appBar: AppBar(
        title: const Text(
          '■■■■',
          style: TextStyle(fontSize: 20, color: Colors.white),
        ),
        backgroundColor: const Color(0xfff4835a),
        foregroundColor: Colors.white,
      ),
      body: RefreshIndicator(
        onRefresh: _loadMyCount,
        child: Container(
          color: const Color(0xfff6f6f6),
          padding: const EdgeInsets.all(16.0),
          child: SingleChildScrollView(
            physics: const AlwaysScrollableScrollPhysics(),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                // User info card
                Container(
                  decoration: BoxDecoration(
                    color: Colors.white,
                    borderRadius: BorderRadius.circular(8.0),
                    boxShadow: [
                      BoxShadow(
                        color: Colors.grey.withValues(alpha: 0.2),
                        spreadRadius: 1,
                        blurRadius: 5,
                        offset: const Offset(0, 2),
                      ),
                    ],
                  ),
                  padding: const EdgeInsets.all(16.0),
```

```
    int r = (hash >> 16) & 0xFF;
    int g = (hash >> 8) & 0xFF;
    int b = hash & 0xFF;
    return Color.fromRGBO(r, g, b, 1.0);
  }

  String _getChannelDisplayName(model.ChatListItem chatItem) {
    if (chatItem.id == -10) return '■■■■';
    if (chatItem.id == -11) return '■■■■■■';
    if (chatItem.id == 0) return '■■■■■';
    if (chatItem.id == -1) return '■■■';
    return chatItem.name;
  }

  String _formatTime(int timestamp) {
    final date = DateTime.fromMillisecondsSinceEpoch(timestamp);
    return DateFormat('MM-dd HH:mm').format(date);
  }

  void _navigateToChatDetail(
    BuildContext context,
    model.ChatListItem chatItem,
  ) {
    if (chatItem.id == null) return;

    // Mark as read
    ref.read(chatProvider.notifier).markAsRead(chatItem.id!);

    if (chatItem.id == -1) {
      // Auction
      context.push('/chat/auction');
    } else if (chatItem.id == -10) {
      // Announcement
      context.push('/chat/group/-10');
    } else if (chatItem.id == -11) {
      // Newbie group
      context.push('/chat/group/-11');
    } else if (chatItem.id == 0) {
      // Lobby
      context.push('/chat/group/0');
    } else if (chatItem.type == model.ChatListItemType.user) {
      // Private chat
      context.push('/chat/private/${chatItem.id}');
    } else {
      // Group chat
      context.push('/chat/group/${chatItem.id}');
    }
  }

  void _showContextMenu(
```

```
    BuildContext context,
    WidgetRef ref,
    model.ChatListItem chatItem,
  ) {
    showAppBottomSheet(
      context: context,
      builder: (context) => AppActionSheet(
        actions: [
          ActionSheetItem(
            title: '■■■■',
            onTap: () => _confirmDelete(context, ref, chatItem),
          ),
        ],
      ),
    );
  }

  void _confirmDelete(
    BuildContext context,
    WidgetRef ref,
    model.ChatListItem chatItem,
  ) {
    showDialog(
      context: context,
      builder: (BuildContext context) {
        return AlertDialog(
          title: const Text('■■'),
          content: const Text('■■■■■■■■■■'),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(),
              child: const Text('■■'),
            ),
            TextButton(
              onPressed: () {
                Navigator.of(context).pop();
                // Delete conversation
                ref.read(chatProvider.notifier).deleteChat(chatItem.id ?? 0);
              },
              child: const Text('■■', style: TextStyle(color: Colors.red)),
            ),
          ],
        );
      },
    );
  }
}

/* === lib/presentation/pages/settings/account_security_page.dart === */
import 'dart:async';
```

```dart
import 'package:flutter/material.dart';
import '../../../core/theme/app_colors.dart';
import '../../../data/models/login_binding_model.dart';
import '../../../data/services/login_binding_service.dart';
import '../../../data/repositories/auth_repository.dart';

class AccountSecurityPage extends StatefulWidget {
  const AccountSecurityPage({super.key});

  @override
  State<AccountSecurityPage> createState() => _AccountSecurityPageState();
}

class _AccountSecurityPageState extends State<AccountSecurityPage> {
  final _loginBindingService = LoginBindingService();
  final _authRepository = AuthRepository();
  List<LoginBindingDto> _bindings = [];
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
    _loadBindings();
  }

  Future<void> _loadBindings() async {
    try {
      final bindings = await _loginBindingService.getLoginBindings();
      if (mounted) {
        setState(() {
          _bindings = bindings;
          _isLoading = false;
        });
      }
    } catch (e) {
      if (mounted) {
        setState(() => _isLoading = false);
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('■■■■■■■■: $e')),
        );
      }
    }
  }

  bool _hasBinding(LoginBindingType type) {
    return _bindings.any((b) => b.bindingType == type);
  }

  LoginBindingDto? _getBinding(LoginBindingType type) {
    try {
```

```
      return _bindings.firstWhere((b) => b.bindingType == type);
    } catch (e) {
      return null;
    }
  }

  Future<void> _showBindPhoneDialog() async {
    final phoneController = TextEditingController();
    final codeController = TextEditingController();
    int countdown = 0;
    Timer? timer;

    await showDialog(
      context: context,
      builder: (context) {
        return StatefulBuilder(
          builder: (context, setDialogState) {
            return AlertDialog(
              title: const Text('■■■■■'),
              content: Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  TextField(
                    controller: phoneController,
                    keyboardType: TextInputType.phone,
                    decoration: InputDecoration(
                      labelText: '■■■',
                      border: OutlineInputBorder(
                        borderRadius: BorderRadius.circular(8),
                      ),
                    ),
                  ),
                  const SizedBox(height: 12),
                  Row(
                    children: [
                      Expanded(
                        child: TextField(
                          controller: codeController,
                          keyboardType: TextInputType.number,
                          decoration: InputDecoration(
                            labelText: '■■■',
                            border: OutlineInputBorder(
                              borderRadius: BorderRadius.circular(8),
                            ),
                          ),
                        ),
                      ),
                      const SizedBox(width: 12),
                      SizedBox(
                        width: 100,
```

```
                        child: TextButton(
                          onPressed: countdown > 0
                              ? null
                              : () async {
                                  final phone = phoneController.text.trim();
                                  if (phone.isEmpty ||
                                      !RegExp(r'^1[3-9]\d{9}$')
                                          .hasMatch(phone)) {
                                    ScaffoldMessenger.of(context).showSnackBar(
                                      const SnackBar(
                                          content: Text('■■■■■■■■■■')),
                                    );
                                    return;
                                  }
                                  try {
                                    await _authRepository.sendSmsCode(
                                      phone,
                                      purpose: 'bindphone',
                                    );
                                    setDialogState(() => countdown = 60);
                                    timer = Timer.periodic(
                                        const Duration(seconds: 1), (t) {
                                      if (countdown > 0) {
                                        setDialogState(() => countdown--);
                                      } else {
                                        t.cancel();
                                      }
                                    });
                                    if (context.mounted) {
                                      ScaffoldMessenger.of(context).showSnackBar(
                                        const SnackBar(content: Text('■■■■■■')),
                                      );
                                    }
                                  } catch (e) {
                                    if (context.mounted) {
                                      ScaffoldMessenger.of(context).showSnackBar(
                                        SnackBar(
                                            content: Text('■■■■: $e')),
                                      );
                                    }
                                  }
                                },
                          style: TextButton.styleFrom(
                            backgroundColor: AppColors.primary.withValues(alpha: 0.1),
                            shape: RoundedRectangleBorder(
                              borderRadius: BorderRadius.circular(8),
                            ),
                          ),
                          child: Text(
                            countdown > 0 ? '${countdown}s' : '■■■■■',
```

```
                            style: TextStyle(
                              fontSize: 13,
                              color: countdown > 0
                                  ? AppColors.textHint
                                  : AppColors.primary,
                            ),
                          ),
                        ),
                      ),
                    ],
                  ),
                ],
              ),
              actions: [
                TextButton(
                  onPressed: () {
                    timer?.cancel();
                    Navigator.of(context).pop();
                  },
                  child: const Text('■■'),
                ),
                TextButton(
                  onPressed: () async {
                    final phone = phoneController.text.trim();
                    final code = codeController.text.trim();
                    if (phone.isEmpty || code.isEmpty) {
                      ScaffoldMessenger.of(context).showSnackBar(
                        const SnackBar(content: Text('■■■■■■■■')),
                      );
                      return;
                    }
                    try {
                      await _loginBindingService.bindPhone(phone, code);
                      timer?.cancel();
                      Navigator.of(context).pop();
                      _loadBindings();
                      if (context.mounted) {
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(content: Text('■■■■')),
                        );
                      }
                    } catch (e) {
                      if (context.mounted) {
                        ScaffoldMessenger.of(context).showSnackBar(
                          SnackBar(content: Text('■■■■: $e')),
                        );
                      }
                    }
                  },
                  child: Text(
```

```
                    '■■',
                    style: TextStyle(color: AppColors.primary),
                  ),
                ),
              ],
            );
          },
        );
      },
    );

    timer?.cancel();
    phoneController.dispose();
    codeController.dispose();
  }

  Future<void> _showUnbindDialog(LoginBindingDto binding) async {
    final confirmed = await showDialog<bool>(
      context: context,
      builder: (context) {
        return AlertDialog(
          title: const Text('■■■■'),
          content: Text('■■■■■${binding.displayName ?? "■■■■■"}■■'),
          actions: [
            TextButton(
              onPressed: () => Navigator.of(context).pop(false),
              child: const Text('■■'),
            ),
            TextButton(
              onPressed: () => Navigator.of(context).pop(true),
              child: const Text('■■', style: TextStyle(color: Colors.red)),
            ),
          ],
        );
      },
    );

    if (confirmed == true && binding.loginProvider != null) {
      try {
        await _loginBindingService.unbindLogin(binding.loginProvider!);
        _loadBindings();
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('■■■■')),
          );
        }
      } catch (e) {
        if (mounted) {
          ScaffoldMessenger.of(context).showSnackBar(
            SnackBar(content: Text('■■■■: $e')),
```

```
          );
        }
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text(
          '■■■■',
          style: TextStyle(fontSize: 20, color: Colors.white),
        ),
        backgroundColor: AppColors.primary,
        foregroundColor: Colors.white,
      ),
      backgroundColor: const Color(0xfff6f6f6),
      body: _isLoading
          ? const Center(child: CircularProgressIndicator())
          : SingleChildScrollView(
              padding: const EdgeInsets.all(16),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const Text(
                    '■■■■',
                    style: TextStyle(
                      fontSize: 16,
                      fontWeight: FontWeight.bold,
                      color: Color(0xff1a1a1a),
                    ),
                  ),
                  const SizedBox(height: 8),
                  Container(
                    decoration: BoxDecoration(
                      color: Colors.white,
                      borderRadius: BorderRadius.circular(8),
                      boxShadow: [
                        BoxShadow(
                          color: Colors.grey.withValues(alpha: 0.2),
                          spreadRadius: 1,
                          blurRadius: 5,
                          offset: const Offset(0, 2),
                        ),
                      ],
                    ),
                    child: Column(
                      children: [
                        _buildBindingTile(
```

```
                          icon: Icons.chat,
                          iconColor: const Color(0xff07c160),
                          title: '■■■■',
                          binding: _getBinding(LoginBindingType.wechat),
                          onBind: null,
                          onUnbind: _hasBinding(LoginBindingType.wechat)
                              ? () => _showUnbindDialog(
                                    _getBinding(LoginBindingType.wechat)!)
                              : null,
                        ),
                        _buildDivider(),
                        _buildBindingTile(
                          icon: Icons.phone_android,
                          iconColor: AppColors.primary,
                          title: '■■■■■',
                          binding: _getBinding(LoginBindingType.phone),
                          onBind: _hasBinding(LoginBindingType.phone)
                              ? null
                              : _showBindPhoneDialog,
                          onUnbind: _hasBinding(LoginBindingType.phone)
                              ? () => _showUnbindDialog(
                                    _getBinding(LoginBindingType.phone)!)
                              : null,
                        ),
                        _buildDivider(),
                        _buildBindingTile(
                          icon: Icons.lock,
                          iconColor: Colors.blue,
                          title: '■■■■',
                          binding: _getBinding(LoginBindingType.password),
                          onBind: null,
                          onUnbind: _hasBinding(LoginBindingType.password)
                              ? () => _showUnbindDialog(
                                    _getBinding(LoginBindingType.password)!)
                              : null,
                        ),
                      ],
                    ),
                  ),
                  const SizedBox(height: 16),
                  const Text(
                    '■■■■■■■■■■■■■■■■■■■■■■■■',
                    style: TextStyle(fontSize: 12, color: AppColors.textSecondary),
                  ),
                ],
              ),
            ),
          );
        }
```

```
  Widget _buildBindingTile({
    required IconData icon,
    required Color iconColor,
    required String title,
    required LoginBindingDto? binding,
    required VoidCallback? onBind,
    required VoidCallback? onUnbind,
  }) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
      child: Row(
        children: [
          Container(
            width: 40,
            height: 40,
            decoration: BoxDecoration(
              color: iconColor.withValues(alpha: 0.1),
              borderRadius: BorderRadius.circular(8),
            ),
            child: Icon(icon, size: 24, color: iconColor),
          ),
          const SizedBox(width: 12),
          Expanded(
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  title,
                  style: const TextStyle(
                    fontSize: 16,
                    color: Color(0xff1a1a1a),
                  ),
                ),
                if (binding != null && binding.providerKey != null)
                  Text(
                    binding.providerKey!,
                    style: const TextStyle(
                      fontSize: 12,
                      color: AppColors.textSecondary,
                    ),
                  ),
              ],
            ),
          ),
          if (binding != null)
            TextButton(
              onPressed: onUnbind,
              child: const Text(
                '■■',
                style: TextStyle(color: Colors.red),
```

```
              ),
            )
          else if (onBind != null)
            TextButton(
              onPressed: onBind,
              child: Text(
                '■■',
                style: TextStyle(color: AppColors.primary),
              ),
            )
          else
            const Text(
              '■■■■',
              style: TextStyle(color: AppColors.textHint),
            ),
        ],
      ),
    );
  }

  Widget _buildDivider() {
    return const Divider(
      height: 1,
      indent: 68,
      endIndent: 16,
      color: Color(0xffeeeeee),
    );
  }
}

/* === lib/presentation/providers/chat_provider.dart === */
import 'dart:async';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../data/services/websocket_service.dart';
import '../../data/api/api_client.dart';
import '../../data/api/api_endpoints.dart';
import '../../data/services/storage_service.dart';

// Chat list item type enum
enum ChatListItemType { group, user, system }

// Chat message type enum
enum ChatMessageType { text, image, file }

// Chat message status enum
enum ChatMessageStatus { sending, sent, delivered, read }

// Chat list item model
class ChatListItem {
  final int? id;
```

```
  final String name;
  final ChatListItemType type;
  final int? time;
  final String? avatar;
  final String? lastMsg;
  final int unread;
  final int order;

  ChatListItem({
    this.id,
    required this.name,
    required this.type,
    this.time,
    this.avatar,
    this.lastMsg,
    required this.unread,
    required this.order,
  });

  factory ChatListItem.fromJson(Map<String, dynamic> json) {
    return ChatListItem(
      id: json['id'],
      name: json['name'] ?? '',
      type: ChatListItemType.values[json['type'] ?? 1], // default to user
      time: json['time'],
      avatar: json['avatar'],
      lastMsg: json['lastMsg'],
      unread: json['unread'] ?? 0,
      order: json['order'] ?? 0,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'name': name,
      'type': type.index,
      'time': time,
      'avatar': avatar,
      'lastMsg': lastMsg,
      'unread': unread,
      'order': order,
    };
  }
}

// Chat message model
class ChatMessage {
  final String? id;
  final ChatMessageType? type;
```

```
  final ChatMessageStatus? status;
  final String? chan;
  final int? from;
  final String? fromName;
  final bool? fromAdmin;
  final String? msg;
  final int? time;
  final String? avatar;
  final dynamic payload;

  ChatMessage({
    this.id,
    this.type,
    this.status,
    this.chan,
    this.from,
    this.fromName,
    this.fromAdmin,
    this.msg,
    this.time,
    this.avatar,
    this.payload,
  });

  factory ChatMessage.fromJson(Map<String, dynamic> json) {
    return ChatMessage(
      id: json['id'],
      type: json['type'] != null ? ChatMessageType.values[json['type']] : null,
      status: json['status'] != null
          ? ChatMessageStatus.values[json['status']]
          : null,
      chan: json['chan'],
      from: json['from'],
      fromName: json['fromName'],
      fromAdmin: json['fromAdmin'],
      msg: json['msg'],
      time: json['time'],
      avatar: json['avatar'],
      payload: json['payload'],
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'type': type?.index,
      'status': status?.index,
      'chan': chan,
      'from': from,
      'fromName': fromName,
```

```
        'fromAdmin': fromAdmin,
        'msg': msg,
        'time': time,
        'avatar': avatar,
        'payload': payload,
      };
    }
  }

  // Chat state
  class ChatState {
    final List<ChatListItem> chatList;
    final bool isWebSocketConnected;
    final int unreadCount;
    final bool isLoading;

    ChatState({
      required this.chatList,
      required this.isWebSocketConnected,
      required this.unreadCount,
      required this.isLoading,
    });

    ChatState copyWith({
      List<ChatListItem>? chatList,
      bool? isWebSocketConnected,
      int? unreadCount,
      bool? isLoading,
    }) {
      return ChatState(
        chatList: chatList ?? this.chatList,
        isWebSocketConnected: isWebSocketConnected ?? this.isWebSocketConnected,
        unreadCount: unreadCount ?? this.unreadCount,
        isLoading: isLoading ?? this.isLoading,
      );
    }

    factory ChatState.initial() {
      return ChatState(
        chatList: [],
        isWebSocketConnected: false,
        unreadCount: 0,
        isLoading: false,
      );
    }
  }

  // WebSocket service provider
  final webSocketServiceProvider = Provider<WebSocketService>((ref) {
    return WebSocketService();
```

```
});

// Chat notifier
class ChatNotifier extends StateNotifier<ChatState> {
  final Ref _ref;
  StreamSubscription<Map<String, dynamic>>? _messageSubscription;

  ChatNotifier(this._ref) : super(ChatState.initial()) {
    _initializeWebSocket();
  }

  Future<void> _initializeWebSocket() async {
    final webSocketService = _ref.read(webSocketServiceProvider);

    // Listen to WebSocket messages
    _messageSubscription = webSocketService.messageStream.listen(
      (message) {
        _handleIncomingMessage(message);
      },
      onError: (error) {
        print('WebSocket error in chat: $error');
      },
    );
  }

  void _handleIncomingMessage(Map<String, dynamic> message) {
    final target = message['target'];

    if (target == 'ReceiveMessage' || target == 'ReceiveChannelMessage') {
      final arguments = message['arguments'] as List<dynamic>?;
      if (arguments != null && arguments.isNotEmpty) {
        final chatMessage = ChatMessage.fromJson(
          arguments.first as Map<String, dynamic>,
        );

        // Update chat list with new message
        final updatedChatList = [...state.chatList];
        final existingIndex = updatedChatList.indexWhere(
          (item) => item.id == chatMessage.from,
        );

        if (existingIndex != -1) {
          final existingItem = updatedChatList[existingIndex];
          updatedChatList[existingIndex] = existingItem.copyWith(
            lastMsg: chatMessage.msg,
            time: chatMessage.time,
            unread: existingItem.unread + 1,
          );
        } else {
          // Add new chat item if not exists
```

```
            updatedChatList.add(
              ChatListItem(
                id: chatMessage.from,
                name: chatMessage.fromName ?? 'Unknown',
                type: ChatListItemType.user,
                time: chatMessage.time,
                avatar: chatMessage.avatar,
                lastMsg: chatMessage.msg,
                unread: 1,
                order: DateTime.now().millisecondsSinceEpoch,
              ),
            );
          }

          // Calculate total unread count
          final totalUnread = updatedChatList.fold(
            0,
            (sum, item) => sum + item.unread,
          );

          state = state.copyWith(
            chatList: updatedChatList,
            unreadCount: totalUnread,
          );
        }
      }
    }

    Future<void> connectWebSocket() async {
      print('========== ChatProvider.connectWebSocket() ■■■ ==========');

      final webSocketService = _ref.read(webSocketServiceProvider);

      // ■■ token
      final storageService = StorageService();
      final token = await storageService.getToken();

      print(
        '[ChatProvider] token: ${token != null ? "■■ (${token.length}■■)" : "■■■"}',
      );

      if (token == null || token.isEmpty) {
        print('[ChatProvider] ■ token■■■ WebSocket ■■');
        return;
      }

      print('[ChatProvider] ■■ webSocketService.connect()...');
      await webSocketService.connect(token: token);

      state = state.copyWith(isWebSocketConnected: webSocketService.isConnected);
```

```
    print('[ChatProvider] WebSocket ■■■■: ${webSocketService.isConnected}');
  }

  Future<void> disconnectWebSocket() async {
    final webSocketService = _ref.read(webSocketServiceProvider);
    await webSocketService.disconnect();

    state = state.copyWith(isWebSocketConnected: false);
  }

  Future<void> loadChatList() async {
    state = state.copyWith(isLoading: true);

    try {
      final storageService = StorageService();
      final token = await storageService.getToken();

      print('loadChatList: token = $token');

      if (token == null || token.isEmpty) {
        print('loadChatList: No token, returning empty list');
        state = state.copyWith(chatList: [], isLoading: false);
        return;
      }

      final response = await ApiClient().dio.get(ApiEndpoints.getChatList);
      print('loadChatList: response.data = ${response.data}');

      List<dynamic>? items;
      if (response.data is List) {
        items = response.data as List;
      } else if (response.data is Map && response.data['items'] != null) {
        items = response.data['items'] as List;
      }

      print('loadChatList: items count = ${items?.length ?? 0}');

      if (items != null) {
        final chatList = items
            .map((json) => ChatListItem.fromJson(json))
            .toList();

        chatList.sort((a, b) => b.order.compareTo(a.order));

        final totalUnread = chatList.fold(0, (sum, item) => sum + item.unread);

        state = state.copyWith(
          chatList: chatList,
          unreadCount: totalUnread,
          isLoading: false,
```

```
        );
      } else {
        state = state.copyWith(chatList: [], isLoading: false);
      }
    } catch (e) {
      print('Error loading chat list: $e');
      state = state.copyWith(chatList: [], isLoading: false);
    }
  }

  Future<void> deleteChat(int chatId) async {
    try {
      // Call API to delete chat
      await ApiClient().dio.delete('${ApiEndpoints.deleteChatList}/$chatId');

      // Remove from local state
      final updatedChatList = state.chatList
          .where((item) => item.id != chatId)
          .toList();
      final totalUnread = updatedChatList.fold(
        0,
        (sum, item) => sum + item.unread,
      );

      state = state.copyWith(
        chatList: updatedChatList,
        unreadCount: totalUnread,
      );
    } catch (e) {
      print('Error deleting chat: $e');
      // Still remove from local state even if API fails
      final updatedChatList = state.chatList
          .where((item) => item.id != chatId)
          .toList();
      final totalUnread = updatedChatList.fold(
        0,
        (sum, item) => sum + item.unread,
      );

      state = state.copyWith(
        chatList: updatedChatList,
        unreadCount: totalUnread,
      );
    }
  }

  Future<void> markAsRead(int chatId) async {
    final updatedChatList = [...state.chatList];
    final index = updatedChatList.indexWhere((item) => item.id == chatId);

```

```
    if (index != -1) {
      updatedChatList[index] = updatedChatList[index].copyWith(unread: 0);
      final totalUnread = updatedChatList.fold(
        0,
        (sum, item) => sum + item.unread,
      );

      state = state.copyWith(
        chatList: updatedChatList,
        unreadCount: totalUnread,
      );
    }
  }

  ChatListItem? getChatById(int chatId) {
    return state.chatList.firstWhere(
      (item) => item.id == chatId,
      orElse: () => state.chatList.first,
    );
  }

  @override
  void dispose() {
    _messageSubscription?.cancel();
    final webSocketService = _ref.read(webSocketServiceProvider);
    webSocketService.dispose();
    super.dispose();
  }
}

// Extension to allow copying ChatListItem with updated values
extension ChatListItemExtension on ChatListItem {
  ChatListItem copyWith({
    int? id,
    String? name,
    ChatListItemType? type,
    int? time,
    String? avatar,
    String? lastMsg,
    int? unread,
    int? order,
  }) {
    return ChatListItem(
      id: id ?? this.id,
      name: name ?? this.name,
      type: type ?? this.type,
      time: time ?? this.time,
      avatar: avatar ?? this.avatar,
      lastMsg: lastMsg ?? this.lastMsg,
      unread: unread ?? this.unread,
```

```
      order: order ?? this.order,
    );
  }
}

// Chat provider
final chatProvider = StateNotifierProvider<ChatNotifier, ChatState>((ref) {
  return ChatNotifier(ref);
});

/* === lib/presentation/pages/chat/private_chat_page.dart === */
import 'dart:io';
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:image_picker/image_picker.dart';
import '../../providers/chat_emoji_store.dart';
import '../../providers/chat_store.dart';
import '../../providers/user_provider.dart';
import '../../../data/models/chat_message_model.dart';
import '../../../data/services/upload_service.dart';
import '../../widgets/chat/messages/message_widget.dart';
import '../../widgets/chat/chat_input_area.dart';

/// ■■■■
/// ■ UniApp chatMain.vue ■■■■
class PrivateChatPage extends ConsumerStatefulWidget {
  final int friendId;
  final String friendName;
  final String? friendAvatar;

  const PrivateChatPage({
    super.key,
    required this.friendId,
    required this.friendName,
    this.friendAvatar,
  });

  @override
  ConsumerState<PrivateChatPage> createState() => _PrivateChatPageState();
}

class _PrivateChatPageState extends ConsumerState<PrivateChatPage> {
  final ScrollController _scrollController = ScrollController();
  final ImagePicker _imagePicker = ImagePicker();
  final UploadService _uploadService = UploadService();
  bool _isUploadingImage = false;

  @override
  void initState() {
    super.initState();
```

```
    _scrollController.addListener(_onScroll);

    // ■■■ - ■ UniApp onLoad ■■
    WidgetsBinding.instance.addPostFrameCallback((_) async {
      // ■■ WebSocket
      await ref.read(chatStoreProvider.notifier).connectServer();

      // ■■■■■■
      ref
          .read(chatStoreProvider.notifier)
          .setCurrentChatId(
            widget.friendId,
            name: widget.friendName,
            isGroup: false,
          );

      // ■■■■
      ref.read(chatStoreProvider.notifier).markAsRead(widget.friendId);

      // ■■■■■■
      await ref
          .read(chatStoreProvider.notifier)
          .getPrivateHistory(widget.friendId);

      // ■■■■■
      _scrollToBottom();
    });
  }

  @override
  void dispose() {
    _scrollController.removeListener(_onScroll);
    _scrollController.dispose();
    super.dispose();
  }

  void _onScroll() {
    if (_scrollController.position.pixels >=
        _scrollController.position.maxScrollExtent - 100) {
      final messages = ref.read(currentChatMessagesProvider);
      final lastTime = messages.isNotEmpty ? messages.last.time : null;
      if (lastTime != null) {
        ref
            .read(chatStoreProvider.notifier)
            .getPrivateHistory(widget.friendId, lastTime: lastTime);
      }
    }
  }

  void _scrollToBottom() {
```

```
    if (_scrollController.hasClients) {
      WidgetsBinding.instance.addPostFrameCallback((_) {
        Future.delayed(const Duration(milliseconds: 50), () {
          if (_scrollController.hasClients &&
              _scrollController.position.maxScrollExtent > 0) {
            _scrollController.animateTo(
              _scrollController.position.maxScrollExtent,
              duration: const Duration(milliseconds: 300),
              curve: Curves.easeOut,
            );
          }
        });
      });
    }
  }

  Future<void> _onSendText(String text) async {
    await ref
        .read(chatStoreProvider.notifier)
        .sendDirectMsg(
          toUserId: widget.friendId,
          message: text,
          type: ChatMessageType.text,
        );
    _scrollToBottom();
  }

  Future<void> _onPickImage() async {
    try {
      final XFile? image = await _imagePicker.pickImage(
        source: ImageSource.gallery,
        maxWidth: 1024,
        maxHeight: 1024,
        imageQuality: 85,
      );

      if (image != null) {
        // ■■■■■■ ID
        final userState = ref.read(userProvider);
        final userId = userState.user?.id;

        // ■■■■■■■
        setState(() {
          _isUploadingImage = true;
        });

        try {
          // ■■■■
          final imageUrl = await _uploadService.uploadImage(
            image.path,
```

```
              userId: userId?.toString(),
            );

            if (imageUrl != null) {
              // ■■■■■■■
              final file = File(image.path);
              final bytes = await file.readAsBytes();
              final decodedImage = await decodeImageFromList(bytes);
              final width = decodedImage.width;
              final height = decodedImage.height;

              // ■■ payload■■ UniApp ■■■■
              final payload = {'url': imageUrl, 'width': width, 'height': height};

              // ■■■■■■■
              await ref
                  .read(chatStoreProvider.notifier)
                  .sendDirectMsg(
                    toUserId: widget.friendId,
                    message: imageUrl,
                    type: ChatMessageType.image,
                    payload: payload,
                  );
              _scrollToBottom();
            } else {
              if (mounted) {
                ScaffoldMessenger.of(
                  context,
                ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■■■')));
              }
            }
          } finally {
            // ■■■■■■■
            if (mounted) {
              setState(() {
                _isUploadingImage = false;
              });
            }
          }
        }
      } catch (e) {
        if (mounted) {
          setState(() {
            _isUploadingImage = false;
          });
          ScaffoldMessenger.of(
            context,
          ).showSnackBar(SnackBar(content: Text('■■■■■■■: $e')));
        }
      }
```

```
  }

  /// ■■■■■■■■■■■■■■■■■
  void _onSelectFavoriteEmoji(dynamic emoji) {
    final url = emoji.url;
    if (url == null || url.isEmpty) return;

    // ■■ payload
    final payload = {'url': url, 'width': 200, 'height': 200};

    // ■■■■■■■
    ref
        .read(chatStoreProvider.notifier)
        .sendDirectMsg(
          toUserId: widget.friendId,
          message: url,
          type: ChatMessageType.image,
          payload: payload,
        );

    _scrollToBottom();
  }

  @override
  Widget build(BuildContext context) {
    final messages = ref.watch(currentChatMessagesProvider);

    return Scaffold(
      appBar: AppBar(
        title: Text(
          widget.friendName,
          style: const TextStyle(fontSize: 20, color: Colors.white),
        ),
        backgroundColor: const Color(0xFFF4835A),
        foregroundColor: Colors.white,
      ),
      body: Stack(
        children: [
          Column(
            children: [
              Expanded(
                child: Container(
                  color: const Color(0xFFFAF1F0),
                  child: messages.isEmpty
                      ? _buildEmptyState()
                      : _buildMessageList(messages),
                ),
              ),
              ChatInputArea(
                onSendText: _onSendText,
```

```
                onSelectImage: _onPickImage,
                showFavoriteTab: true,
                onSelectFavoriteEmoji: _onSelectFavoriteEmoji,
              ),
            ],
          ),
          // ■■■■
          if (_isUploadingImage) _buildLoadingOverlay(),
        ],
      ),
    );
  }

  Widget _buildLoadingOverlay() {
    return Container(
      color: Colors.black.withOpacity(0.3),
      child: Center(
        child: Container(
          padding: const EdgeInsets.symmetric(horizontal: 24, vertical: 20),
          decoration: BoxDecoration(
            color: Colors.white,
            borderRadius: BorderRadius.circular(12),
          ),
          child: Column(
            mainAxisSize: MainAxisSize.min,
            children: [
              const SizedBox(
                width: 32,
                height: 32,
                child: CircularProgressIndicator(
                  strokeWidth: 3,
                  valueColor: AlwaysStoppedAnimation<Color>(Color(0xFFF4835A)),
                ),
              ),
              const SizedBox(height: 16),
              Text(
                '■■■■■■...',
                style: TextStyle(fontSize: 14, color: Colors.grey.shade700),
              ),
            ],
          ),
        ),
      ),
    );
  }

  Widget _buildEmptyState() {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
```

```
        children: [
          Icon(Icons.person, size: 64, color: Colors.grey.shade400),
          const SizedBox(height: 16),
          Text(
            '■■■■',
            style: TextStyle(fontSize: 16, color: Colors.grey.shade500),
          ),
          const SizedBox(height: 8),
          Text(
            '■■■■■■■■■■■■',
            style: TextStyle(fontSize: 14, color: Colors.grey.shade400),
          ),
        ],
      ),
    );
  }

  Widget _buildMessageList(List<ChatMessage> messages) {
    return ListView.builder(
      controller: _scrollController,
      padding: const EdgeInsets.all(16),
      itemCount: messages.length,
      itemBuilder: (context, index) {
        final message = messages[index];
        return _buildMessageItem(message);
      },
    );
  }

  Widget _buildMessageItem(ChatMessage message) {
    final isSelf = message.from != null && message.from == _getCurrentUserId();

    if (message.type == ChatMessageType.welcome ||
        message.type == ChatMessageType.banUser ||
        message.type == ChatMessageType.backout) {
      return Padding(
        padding: const EdgeInsets.symmetric(vertical: 8),
        child: MessageWidget(message: message),
      );
    }

    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 8),
      child: Row(
        mainAxisAlignment: isSelf
            ? MainAxisAlignment.end
            : MainAxisAlignment.start,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          if (!isSelf) ...[_buildAvatar(message), const SizedBox(width: 10)],
```

```
          Flexible(
            child: Column(
              crossAxisAlignment: isSelf
                  ? CrossAxisAlignment.end
                  : CrossAxisAlignment.start,
              children: [
                MessageWidget(
                  message: message,
                  onTap: () => debugPrint('■■■■■: ${message.id}'),
                ),
              ],
            ),
          ),
          if (isSelf) ...[const SizedBox(width: 10), _buildAvatar(message)],
        ],
      ),
    );
  }

  int? _getCurrentUserId() {
    final userState = ref.read(userProvider);
    return userState.user?.id;
  }

  Widget _buildAvatar(ChatMessage message) {
    // ■■■■■■■■ avatar ■■
    String? avatarUrl;
    if (message.from != _getCurrentUserId() && message.avatar != null) {
      final avatar = message.avatar!;
      avatarUrl = avatar.startsWith('http')
          ? avatar
          : 'https://image.molitao.top/$avatar';
    }
    // ■■■■ message.avatar■■■■■■ friendAvatar ■■■■
    avatarUrl ??= widget.friendAvatar;

    if (avatarUrl == null || avatarUrl.isEmpty) {
      // ■■■■■■■■■■
      return Container(
        width: 36,
        height: 36,
        decoration: const BoxDecoration(
          color: Color(0xFFF4835A),
          shape: BoxShape.circle,
        ),
        child: Center(
          child: Text(
            _getAvatarText(message.fromName ?? '■■'),
            style: const TextStyle(
              color: Colors.white,
```

```
              fontSize: 12,
              fontWeight: FontWeight.bold,
            ),
          ),
        ),
      );
    }

    // ■■■■■■
    return Container(
      width: 36,
      height: 36,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        image: DecorationImage(
          image: NetworkImage(avatarUrl),
          fit: BoxFit.cover,
        ),
      ),
    );
  }

  String _getAvatarText(String name) {
    if (name.isEmpty) return '■';
    return name.length > 2 ? name.substring(0, 2) : name;
  }
}

/* === lib/presentation/pages/chat/chat_message_preview_page.dart === */
import 'package:flutter/material.dart';
import 'package:molitao_app/data/models/chat_message_model.dart';
import 'package:molitao_app/presentation/widgets/chat/messages/message_widget.dart';
import 'package:molitao_app/presentation/widgets/chat/chat_input_area.dart';

/// ■■■■■■■■
/// ■■■■■■■■■■■■■■ UI ■■
class ChatMessagePreviewPage extends StatefulWidget {
  const ChatMessagePreviewPage({Key? key}) : super(key: key);

  @override
  State<ChatMessagePreviewPage> createState() => _ChatMessagePreviewPageState();
}

class _ChatMessagePreviewPageState extends State<ChatMessagePreviewPage> {
  final List<ChatMessage> _messages = List.from(mockMessages);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
```

```
        title: const Text(
          '■■■■■■',
          style: TextStyle(fontSize: 20, color: Colors.white),
        ),
        backgroundColor: const Color(0xFFF4835A),
        foregroundColor: Colors.white,
      ),
      body: Column(
        children: [
          // ■■■■
          Expanded(
            child: Container(
              color: const Color(0xFFFAF1F0),
              child: ListView.builder(
                padding: const EdgeInsets.all(16),
                itemCount: _messages.length,
                itemBuilder: (context, index) {
                  final message = _messages[index];
                  return _buildMessageItem(message);
                },
              ),
            ),
          ),

          // ■■■■
          ChatInputArea(
            onSendText: _handleSendText,
            onSelectImage: _handleSelectImage,
            onSelectEmoji: _handleSelectEmoji,
          ),
        ],
      ),
    );
  }

  void _handleSendText(String text) {
    // ■■■■■
    final newMessage = ChatMessage(
      id: DateTime.now().millisecondsSinceEpoch.toString(),
      type: ChatMessageType.text,
      from: 1001,
      fromName: '■',
      msg: text,
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    );

    setState(() {
      _messages.add(newMessage);
    });
  }
```

```
3951
3952    void _handleSelectImage() {
3953      // ■■■■■■■
3954      final newMessage = ChatMessage(
3955        id: DateTime.now().millisecondsSinceEpoch.toString(),
3956        type: ChatMessageType.image,
3957        from: 1001,
3958        fromName: '■',
3959        msg: 'https://picsum.photos/300/300',
3960        payload: {'url': 'https://picsum.photos/300/300'},
3961        time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
3962      );
3963
3964      setState(() {
3965        _messages.add(newMessage);
3966      });
3967
3968      ScaffoldMessenger.of(
3969        context,
3970      ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■')));
3971    }
3972
3973    void _handleSelectEmoji(String emojiCode) {
3974      debugPrint('■■■■■: $emojiCode');
3975    }
3976
3977    Widget _buildMessageItem(ChatMessage message) {
3978      // ■■■■■■■■■■■■
3979      final isSelf = message.from == 1001;
3980
3981      // ■■■■■■■■■■■■■■
3982      if (message.type == ChatMessageType.welcome ||
3983          message.type == ChatMessageType.banUser ||
3984          message.type == ChatMessageType.backout) {
3985        return Padding(
3986          padding: const EdgeInsets.symmetric(vertical: 8),
3987          child: MessageWidget(message: message),
3988        );
3989      }
3990
3991      return Padding(
3992        padding: const EdgeInsets.symmetric(vertical: 8),
3993        child: Row(
3994          mainAxisAlignment: isSelf
3995              ? MainAxisAlignment.end
3996              : MainAxisAlignment.start,
3997          crossAxisAlignment: CrossAxisAlignment.start,
3998          children: [
3999            // ■■■■■■■■■■■■■■■
4000            if (!isSelf) ...[_buildAvatar(message), const SizedBox(width: 10)],
```

```
          // ■■■■
          Flexible(
            child: Column(
              crossAxisAlignment: isSelf
                  ? CrossAxisAlignment.end
                  : CrossAxisAlignment.start,
              children: [
                // ■■■■■■
                if (!isSelf) _buildUserName(message),
                const SizedBox(height: 4),
                // ■■■■
                MessageWidget(
                  message: message,
                  onTap: () => _handleMessageTap(message),
                ),
              ],
            ),
          ),
          // ■■■■■■■■■■■■
          if (isSelf) ...[const SizedBox(width: 10), _buildAvatar(message)],
        ],
      ),
    );
  }

  Widget _buildAvatar(ChatMessage message) {
    return Container(
      width: 40,
      height: 40,
      decoration: BoxDecoration(
        color: _getAvatarColor(message.from ?? 0),
        shape: BoxShape.circle,
      ),
      child: Center(
        child: Text(
          _getAvatarText(message.fromName ?? '■■'),
          style: const TextStyle(
            color: Colors.white,
            fontSize: 14,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    );
  }

  Widget _buildUserName(ChatMessage message) {
    return Row(
      mainAxisSize: MainAxisSize.min,
      children: [
```

```
        if (message.fromAdmin == true) ...[
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
            decoration: BoxDecoration(
              color: Colors.red,
              borderRadius: BorderRadius.circular(4),
            ),
            child: const Text(
              '■■',
              style: TextStyle(
                color: Colors.white,
                fontSize: 10,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          const SizedBox(width: 6),
        ],
        if (message.fromTag != null) ...[
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 6, vertical: 2),
            decoration: BoxDecoration(
              color: Colors.orange,
              borderRadius: BorderRadius.circular(4),
            ),
            child: Text(
              message.fromTag!,
              style: const TextStyle(
                color: Colors.white,
                fontSize: 10,
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
          const SizedBox(width: 6),
        ],
        Text(
          message.fromName ?? '■■■■',
          style: const TextStyle(fontSize: 12, color: Color(0xFF999999)),
        ),
      ],
    );
  }

  void _handleMessageTap(ChatMessage message) {
    debugPrint('■■■■■: ${message.id}, ■■: ${message.type}');
  }

  Color _getAvatarColor(int userId) {
    final colors = [
```

```
      const Color(0xFFF4835A),
      const Color(0xFF1890FF),
      const Color(0xFFFF4D4F),
      const Color(0xFF722ED1),
      const Color(0xFF52C41A),
      const Color(0xFFFA8C16),
    ];
    return colors[userId % colors.length];
  }

  String _getAvatarText(String name) {
    if (name.isEmpty) return '■';
    return name.length > 2 ? name.substring(0, 2) : name;
  }
}

/// Mock ■■■■
/// ■■■■■■■■■■■
final List<ChatMessage> mockMessages = [
  // ■■■■
  ChatMessage(
    id: '1',
    type: ChatMessageType.welcome,
    fromName: '■■',
    msg: '■■■■■■■■■■■■■■■■■■',
    time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
  ),

  // ■■■■■■■■■
  ChatMessage(
    id: '2',
    type: ChatMessageType.text,
    from: 1002,
    fromName: '■■',
    msg: '■■■■■■■■[■■]■■■■■■■[■■]',
    time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
  ),

  // ■■■■■■■
  ChatMessage(
    id: '3',
    type: ChatMessageType.auctionStart,
    from: 1003,
    fromName: '■■■■',
    fromAdmin: true,
    fromTag: '■■',
    payload: {
      'id': 12345,
      'name': '175■■■■■■■■',
      'description': '■■■: 50■■■\n■■■■■■■■■■',
```

```
    },
    time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
  ),

  // ■■■■
  ChatMessage(
    id: '4',
    type: ChatMessageType.auctionBid,
    from: 1004,
    fromName: '■■',
    payload: {'id': 12345, 'name': '175■■■■■■■', 'currentPrice': 68.0},
    time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
  ),

  // ■■■■2
  ChatMessage(
    id: '5',
    type: ChatMessageType.auctionBid,
    from: 1005,
    fromName: '■■',
    payload: {'id': 12345, 'name': '175■■■■■■■', 'currentPrice': 85.0},
    time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
  ),

  // ■■■■■■ - ■■
  ChatMessage(
    id: '6',
    type: ChatMessageType.kasecStatusChanged,
    from: 1003,
    fromName: '■■■■',
    fromAdmin: true,
    fromTag: '■■',
    msg: '■■■■■■■■■■■■■■■■■■■■',
    payload: {'isKasec': true},
    time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
  ),

  // ■■■■■■
  ChatMessage(
    id: '7',
    type: ChatMessageType.auctionEnd,
    from: 1003,
    fromName: '■■■■',
    fromAdmin: true,
    fromTag: '■■',
    payload: {
      'id': 12345,
      'name': '175■■■■■■■',
      'dealUserName': '■■',
      'finalPrice': 85.0,
```

```
        'status': '■■■',
        'dealTime': DateTime.now().toIso8601String(),
      },
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    ),

    // ■■■■■■■■■■■■
    ChatMessage(
      id: '8',
      type: ChatMessageType.auctionDeal,
      from: 1001,
      fromName: '■■■■■',
      payload: {
        'id': 12345,
        'name': '175■■■■■■■',
        'finalPrice': 85.0,
        'dealTime': DateTime.now().toIso8601String(),
      },
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    ),

    // ■■■■ - ■■
    ChatMessage(
      id: '9',
      type: ChatMessageType.banUser,
      fromName: '■■',
      msg: '■■ ■■ ■■■■ 60 ■■',
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    ),

    // ■■■■■■■■■■
    ChatMessage(
      id: '10',
      type: ChatMessageType.text,
      from: 1001,
      fromName: '■',
      msg: '■■■■■■■■[OK]',
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    ),

    // ■■■■
    ChatMessage(
      id: '11',
      type: ChatMessageType.image,
      from: 1002,
      fromName: '■■',
      msg: 'https://picsum.photos/300/300',
      payload: {'url': 'https://picsum.photos/300/300'},
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    ),
```

```
    // ■■■■■■ - ■■
    ChatMessage(
      id: '12',
      type: ChatMessageType.kasecStatusChanged,
      from: 1003,
      fromName: '■■■■',
      fromAdmin: true,
      fromTag: '■■',
      msg: '■■■■■■■■■■■■■■■■■■■■',
      payload: {'isKasec': false},
      time: DateTime.now().millisecondsSinceEpoch ~/ 1000,
    ),
  ];

  /* === lib/presentation/pages/trading_post/add_post_page.dart === */
  import 'package:flutter/material.dart';
  import 'package:flutter_riverpod/flutter_riverpod.dart';
  import 'package:image_picker/image_picker.dart';
  import '../../../data/repositories/post_repository.dart';
  import '../../../data/services/upload_service.dart';

  /// ■■/■■■■■■
  class AddPostPage extends ConsumerStatefulWidget {
    final int? postId;

    const AddPostPage({super.key, this.postId});

    @override
    ConsumerState<AddPostPage> createState() => _AddPostPageState();
  }

  class _AddPostPageState extends ConsumerState<AddPostPage> {
    final _formKey = GlobalKey<FormState>();
    final _titleController = TextEditingController();
    final _contentController = TextEditingController();
    final _wechatController = TextEditingController();
    final _qqController = TextEditingController();
    final ImagePicker _imagePicker = ImagePicker();
    final UploadService _uploadService = UploadService();

    bool _isLoading = false;
    bool _isUploadingImage = false;
    bool _isEditing = false;
    List<String> _selectedCategories = [];

    final List<Map<String, dynamic>> _categories = [
      {'id': 1, 'name': '■■'},
      {'id': 2, 'name': '■■'},
      {'id': 3, 'name': '■■'},
```

```
    {'id': 4, 'name': '■■'},
    {'id': 5, 'name': '■■'},
  ];

  @override
  void initState() {
    super.initState();
    if (widget.postId != null) {
      _isEditing = true;
      _loadPostData();
    }
  }

  @override
  void dispose() {
    _titleController.dispose();
    _contentController.dispose();
    _wechatController.dispose();
    _qqController.dispose();
    super.dispose();
  }

  Future<void> _loadPostData() async {
    setState(() => _isLoading = true);

    try {
      final post = await PostRepository().getPostDetail(widget.postId!);
      if (post != null && mounted) {
        _titleController.text = post.title ?? '';
        _contentController.text = post.content ?? '';
        _wechatController.text = post.wechat ?? '';
        _qqController.text = post.qq ?? '';

        if (post.categoryName != null) {
          _selectedCategories = post.categoryName!
              .split(',')
              .where((t) => t.isNotEmpty)
              .toList();
        }
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text('■■■■■■: $e')));
      }
    }

    setState(() => _isLoading = false);
  }
```

```
4351
4352    Future<void> _pickImage() async {
4353      try {
4354        final XFile? image = await _imagePicker.pickImage(
4355          source: ImageSource.gallery,
4356          maxWidth: 1024,
4357          maxHeight: 1024,
4358          imageQuality: 85,
4359        );
4360
4361        if (image != null) {
4362          // ■■■■
4363          setState(() => _isUploadingImage = true);
4364
4365          try {
4366            final imageUrl = await _uploadService.uploadImage(image.path);
4367
4368            if (imageUrl != null) {
4369              // ■■■■ HTML ■■■■
4370              final imageHtml = '<img src="$imageUrl" />';
4371              _contentController.text += imageHtml;
4372
4373              if (mounted) {
4374                ScaffoldMessenger.of(
4375                  context,
4376                ).showSnackBar(const SnackBar(content: Text('■■■■■■')));
4377              }
4378            } else {
4379              if (mounted) {
4380                ScaffoldMessenger.of(
4381                  context,
4382                ).showSnackBar(const SnackBar(content: Text('■■■■■■■■■■')));
4383              }
4384            }
4385          } catch (uploadError) {
4386            if (mounted) {
4387              ScaffoldMessenger.of(
4388                context,
4389              ).showSnackBar(SnackBar(content: Text('■■■■■■■$uploadError')));
4390              }
4391          } finally {
4392            if (mounted) {
4393              setState(() => _isUploadingImage = false);
4394            }
4395          }
4396        }
4397      } catch (e) {
4398        if (mounted) {
4399          ScaffoldMessenger.of(
4400            context,
```

```
        ).showSnackBar(SnackBar(content: Text('■■■■■■■$e')));
      }
    }
  }

  Future<void> _submit() async {
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isLoading = true);

    try {
      final repository = PostRepository();
      final categoryIds = _selectedCategories.isNotEmpty
          ? _categories
                .where((c) => _selectedCategories.contains(c['name']))
                .map((c) => c['id'] as int)
                .toList()
          : null;

      if (_isEditing) {
        await repository.updatePost(
          id: widget.postId!,
          title: _titleController.text,
          content: _contentController.text,
          categoryId: categoryIds?.first,
        );
      } else {
        await repository.createPost(
          title: _titleController.text,
          content: _contentController.text,
          categoryId: categoryIds?.first,
        );
      }

      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text(_isEditing ? '■■■■' : '■■■■')));
        Navigator.pop(context, true);
      }
    } catch (e) {
      if (mounted) {
        ScaffoldMessenger.of(
          context,
        ).showSnackBar(SnackBar(content: Text('■■■■: $e')));
      }
    }

    setState(() => _isLoading = false);
  }
```

```
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(
          _isEditing ? '■■■■' : '■■■■',
          style: const TextStyle(fontSize: 20, color: Colors.white),
        ),
        backgroundColor: const Color(0xFFf4835a),
        foregroundColor: Colors.white,
        actions: [
          TextButton(
            onPressed: _isLoading ? null : _submit,
            child: const Text(
              '■■',
              style: TextStyle(color: Colors.white, fontSize: 16),
            ),
          ),
        ],
      ),
      body: _isLoading
          ? const Center(child: CircularProgressIndicator())
          : _buildForm(),
    );
  }

  Widget _buildForm() {
    return Form(
      key: _formKey,
      child: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          // ■■■■
          TextFormField(
            controller: _titleController,
            decoration: const InputDecoration(
              labelText: '■■',
              hintText: '■■■■■',
              border: OutlineInputBorder(),
            ),
            maxLength: 100,
            validator: (value) {
              if (value == null || value.trim().isEmpty) {
                return '■■■■■';
              }
              return null;
            },
          ),
          const SizedBox(height: 16),
```

```
4501
4502            // ■■■■
4503            _buildCategorySelector(),
4504            const SizedBox(height: 16),
4505
4506            // ■■■■
4507            TextFormField(
4508              controller: _contentController,
4509              decoration: InputDecoration(
4510                labelText: '■■',
4511                hintText: '■■■■■',
4512                border: const OutlineInputBorder(),
4513                suffixIcon: _isUploadingImage
4514                    ? const Padding(
4515                        padding: EdgeInsets.all(8.0),
4516                        child: SizedBox(
4517                          width: 20,
4518                          height: 20,
4519                          child: CircularProgressIndicator(strokeWidth: 2),
4520                        ),
4521                      )
4522                    : IconButton(
4523                        icon: const Icon(Icons.image),
4524                        onPressed: _pickImage,
4525                        tooltip: '■■■■',
4526                      ),
4527              ),
4528              maxLines: 8,
4529              maxLength: 5000,
4530              validator: (value) {
4531                if (value == null || value.trim().isEmpty) {
4532                  return '■■■■■';
4533                }
4534                return null;
4535              },
4536            ),
4537            const SizedBox(height: 16),
4538
4539            // ■■■■
4540            _buildContactSection(),
4541          ],
4542        ),
4543      );
4544    }
4545
4546    Widget _buildCategorySelector() {
4547      return Column(
4548        crossAxisAlignment: CrossAxisAlignment.start,
4549        children: [
4550          const Text(
```

```
            '■■',
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.w500),
          ),
          const SizedBox(height: 8),
          Wrap(
            spacing: 8,
            runSpacing: 8,
            children: _categories.map((category) {
              final isSelected = _selectedCategories.contains(category['name']);
              return FilterChip(
                label: Text(category['name']),
                selected: isSelected,
                onSelected: (selected) {
                  setState(() {
                    if (selected) {
                      _selectedCategories.add(category['name']);
                    } else {
                      _selectedCategories.remove(category['name']);
                    }
                  });
                },
                selectedColor: const Color(0xFFf4835a).withOpacity(0.2),
                checkmarkColor: const Color(0xFFf4835a),
              );
            }).toList(),
          ),
        ],
      );
    }

    Widget _buildContactSection() {
      return Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text(
            '■■■■■■■■',
            style: TextStyle(fontSize: 16, fontWeight: FontWeight.w500),
          ),
          const SizedBox(height: 12),
          Row(
            children: [
              Icon(Icons.wechat, color: Colors.green.shade600, size: 20),
              const SizedBox(width: 8),
              Expanded(
                child: TextFormField(
                  controller: _wechatController,
                  decoration: const InputDecoration(
                    labelText: '■■■',
                    border: OutlineInputBorder(),
                    contentPadding: EdgeInsets.symmetric(
```

```
                      horizontal: 12,
                      vertical: 12,
                    ),
                  ),
                ),
              ),
            ],
          ),
          const SizedBox(height: 12),
          Row(
            children: [
              Icon(Icons.chat, color: Colors.blue.shade600, size: 20),
              const SizedBox(width: 8),
              Expanded(
                child: TextFormField(
                  controller: _qqController,
                  decoration: const InputDecoration(
                    labelText: 'QQ■',
                    border: OutlineInputBorder(),
                    contentPadding: EdgeInsets.symmetric(
                      horizontal: 12,
                      vertical: 12,
                    ),
                  ),
                  keyboardType: TextInputType.number,
                ),
              ),
            ],
          ),
        ],
      );
  }
}

/* === lib/presentation/providers/user_provider.dart === */
import 'package:flutter_riverpod/flutter_riverpod.dart';
import '../../data/services/storage_service.dart';
import '../../data/services/push_service.dart';
import '../../data/repositories/auth_repository.dart';

// User data model
class User {
  final int? id;
  final String? userName;
  final String? fullName;
  final String? phoneNumber;
  final String? headImgUrl;
  final double? depositBalance;
  final List<String>? permissions;
  final List<String>? roleNames;
```

```
  final String? qq;
  final String? wx;

  User({
    this.id,
    this.userName,
    this.fullName,
    this.phoneNumber,
    this.headImgUrl,
    this.depositBalance,
    this.permissions,
    this.roleNames,
    this.qq,
    this.wx,
  });

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'],
      userName: json['userName'],
      fullName: json['fullName'] ?? json['name'],
      phoneNumber: json['phoneNumber'],
      headImgUrl: json['headImgUrl'],
      depositBalance: json['depositBalance']?.toDouble(),
      permissions: (json['permissions'] as List<dynamic>?)
          ?.map((e) => e.toString())
          .toList(),
      roleNames: (json['roleNames'] as List<dynamic>?)
          ?.map((e) => e.toString())
          .toList(),
      qq: json['qq'],
      wx: json['wx'],
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'userName': userName,
      'fullName': fullName,
      'phoneNumber': phoneNumber,
      'headImgUrl': headImgUrl,
      'depositBalance': depositBalance,
      'permissions': permissions,
      'roleNames': roleNames,
      'qq': qq,
      'wx': wx,
    };
  }

```

```
  User copyWith({
    int? id,
    String? userName,
    String? fullName,
    String? phoneNumber,
    String? headImgUrl,
    double? depositBalance,
    List<String>? permissions,
    List<String>? roleNames,
    String? qq,
    String? wx,
  }) {
    return User(
      id: id ?? this.id,
      userName: userName ?? this.userName,
      fullName: fullName ?? this.fullName,
      phoneNumber: phoneNumber ?? this.phoneNumber,
      headImgUrl: headImgUrl ?? this.headImgUrl,
      depositBalance: depositBalance ?? this.depositBalance,
      permissions: permissions ?? this.permissions,
      roleNames: roleNames ?? this.roleNames,
      qq: qq ?? this.qq,
      wx: wx ?? this.wx,
    );
  }
}

// User state
class UserState {
  final bool isLoggedIn;
  final String? token;
  final User? user;
  final bool isLoading;
  final List<String>? roles;

  UserState({
    required this.isLoggedIn,
    this.token,
    this.user,
    required this.isLoading,
    this.roles,
  });

  UserState copyWith({
    bool? isLoggedIn,
    String? token,
    User? user,
    bool? isLoading,
    List<String>? roles,
  }) {
```

```
    return UserState(
      isLoggedIn: isLoggedIn ?? this.isLoggedIn,
      token: token ?? this.token,
      user: user ?? this.user,
      isLoading: isLoading ?? this.isLoading,
      roles: roles ?? this.roles,
    );
  }

  factory UserState.initial() {
    return UserState(isLoggedIn: false, isLoading: false);
  }
}

// Storage service provider
final storageServiceProvider = Provider<StorageService>((ref) {
  return StorageService();
});

// Auth repository provider
final authRepositoryProvider = Provider<AuthRepository>((ref) {
  return AuthRepository();
});

// User notifier
class UserNotifier extends StateNotifier<UserState> {
  final Ref _ref;

  UserNotifier(this._ref) : super(UserState.initial()) {
    _initializeUser();
  }

  /// ■■■■■■■■■■■■
  Future<void> _initializeUser() async {
    final storageService = _ref.read(storageServiceProvider);

    final token = await storageService.getToken();
    print('[UserProvider] _initializeUser: token■■=${token != null}');

    if (token != null && token.isNotEmpty) {
      // ■■■■■■■■■■■■■■■■■■■■
      final userData = await storageService.getUserData();
      User? cachedUser;
      if (userData != null) {
        cachedUser = User.fromJson(userData);
        state = state.copyWith(
          token: token,
          isLoggedIn: true,
          user: cachedUser,
        );
```

```
4801        }
4802
4803        // ■■ token ■■■■■■■■■■■■
4804        try {
4805          print('[UserProvider] ■■ token ■■■■■■■...');
4806          final authRepository = _ref.read(authRepositoryProvider);
4807          final response = await authRepository.getCurrentLoginInformations();
4808
4809          if (response != null && response['user'] != null) {
4810            final user = User.fromJson(response['user'] as Map<String, dynamic>);
4811            final roles = (response['roles'] as List<dynamic>?)
4812                ?.map((e) => e.toString())
4813                .toList();
4814
4815            print('[UserProvider] ■■■■■■■:');
4816            print('[UserProvider] - user.id: ${user.id}');
4817            print('[UserProvider] - user.userName: ${user.userName}');
4818            print('[UserProvider] - user.fullName: ${user.fullName}');
4819            print('[UserProvider] - user.headImgUrl: ${user.headImgUrl}');
4820            print('[UserProvider] - roles: $roles');
4821
4822            // ■■■■■■■■■■
4823            await storageService.setUserData(user.toJson());
4824            print('[UserProvider] ■■■■■■■■■');
4825
4826            state = state.copyWith(
4827              token: token,
4828              isLoggedIn: true,
4829              user: user,
4830              roles: roles,
4831            );
4832
4833            print('[UserProvider] state ■■■:');
4834            print('[UserProvider] - state.isLoggedIn: ${state.isLoggedIn}');
4835            print(
4836              '[UserProvider] - state.user?.userName: ${state.user?.userName}',
4837            );
4838            print('[UserProvider] ■■■■■■■: ${user.userName}');
4839
4840            final alias = 'user_${user.id}';
4841            print('[Push] ■■■■■: $alias');
4842            PushService().setAlias(alias);
4843          } else {
4844            // token ■■■■■■■■■■
4845            print('[UserProvider] token ■■■■■■■■■■');
4846            _logoutInternal();
4847          }
4848        } catch (e) {
4849          print('[UserProvider] ■■■■■■■: $e');
4850          // API ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

```
        if (cachedUser != null) {
          print('[UserProvider] ■■■■■■■■■■■■');
          state = state.copyWith(
            token: token,
            isLoggedIn: true,
            user: cachedUser,
          );
        }
      }
    }
  }

  /// ■■■■■■■■■■■■
  Future<bool> login(String token, User user, {List<String>? roles}) async {
    print('[UserProvider] login() ■■■');
    print('[UserProvider] - token: ${token.substring(0, 20)}...');
    print('[UserProvider] - user.id: ${user.id}');
    print('[UserProvider] - user.userName: ${user.userName}');
    print('[UserProvider] - user.fullName: ${user.fullName}');

    state = state.copyWith(isLoading: true);

    try {
      final storageService = _ref.read(storageServiceProvider);
      await storageService.setToken(token);
      await storageService.setUserData(user.toJson());

      state = state.copyWith(
        isLoggedIn: true,
        token: token,
        user: user,
        roles: roles,
        isLoading: false,
      );

      print('[UserProvider] login() ■■');
      print('[UserProvider] - state.isLoggedIn: ${state.isLoggedIn}');
      print('[UserProvider] - state.user: ${state.user?.userName}');

      if (user.id != null) {
        final alias = 'user_${user.id}';
        print('[Push] ■■■■: $alias');
        PushService().setAlias(alias);
      }

      return true;
    } catch (e) {
      print('[UserProvider] login() ■■: $e');
      state = state.copyWith(isLoading: false);
      return false;
```

```
    }
  }

  /// ■■■■■■■
  /// [forceCheck] ■■■ true■■■■ API ■■ token ■■■■
  Future<bool> checkLogin({bool forceCheck = true}) async {
    final token = state.token;

    if (token == null || token.isEmpty) {
      _logoutInternal();
      return false;
    }

    // ■■■■■■■■■■■■■■■■■■■■■■
    if (!forceCheck && state.user?.id != null) {
      return true;
    }

    try {
      final authRepository = _ref.read(authRepositoryProvider);
      final response = await authRepository.getCurrentLoginInformations();

      if (response != null && response['user'] != null) {
        final user = User.fromJson(response['user'] as Map<String, dynamic>);
        final roles = (response['roles'] as List<dynamic>?)
            ?.map((e) => e.toString())
            .toList();

        // ■■■■■■■
        final storageService = _ref.read(storageServiceProvider);
        await storageService.setUserData(user.toJson());

        state = state.copyWith(user: user, roles: roles, isLoggedIn: true);
        return true;
      } else {
        _logoutInternal();
        return false;
      }
    } catch (e) {
      _logoutInternal();
      return false;
    }
  }

  /// ■■■■
  Future<void> logout() async {
    state = state.copyWith(isLoading: true);

    PushService().deleteAlias();

```

```
    try {
      final authRepository = _ref.read(authRepositoryProvider);
      await authRepository.logout();
    } catch (e) {
      // ■■ API ■■■■■■■■■■■■
    }

    state = state.copyWith(
      isLoggedIn: false,
      token: null,
      user: null,
      roles: null,
      isLoading: false,
    );
  }

  void _logoutInternal() {
    state = state.copyWith(
      isLoggedIn: false,
      token: null,
      user: null,
      roles: null,
    );
    StorageService().clearToken();
    StorageService().clearUserData();
  }

  /// ■■■■■■■
  Future<void> updateUser(User user) async {
    state = state.copyWith(user: user);

    try {
      final storageService = _ref.read(storageServiceProvider);
      await storageService.setUserData(user.toJson());
    } catch (e) {
      // ■■■■
    }
  }

  /// ■■■■■■■■■■■■■■ API■
  bool get isAuthenticated => state.isLoggedIn && state.token != null;

  /// ■■■■■■■■■
  bool get isAdmin => state.roles?.contains('Admin') ?? false;
}

// User provider
final userProvider = StateNotifierProvider<UserNotifier, UserState>((ref) {
  return UserNotifier(ref);
});
```